

Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING (SCOPE)

VIT, VELLORE

PROJECT REPORT

ON

SentinelStream: Real-Time Card Fraud Detection and Fraud-Ring Discovery Using Polyglot NoSQL Persistence

Submitted in partial fulfilment of the requirements for the course

BCSE406L – NoSQL Databases

Submitted by

Batch ID: NS14

Team Name: SentinelStream

1. Yash Sehrawat – 23BCE0161

Under the guidance of

Dr. D. Vivek

Fall Semester 2026–2027

BONAFIDE CERTIFICATE

This is to certify that the project report entitled “SentinelStream: Real-Time Card Fraud Detection and Fraud-Ring Discovery Using Polyglot NoSQL Persistence” is the Bonafide work carried out by the following student as part of the course BCSE406L – NoSQL Databases during the Fall Semester 2026–2027, under the guidance of Dr. D. Vivek, School of Computer Science and Engineering (SCOPE), VIT, Vellore.

S. No.	**Register Number**	**Student Name**
1	23BCE0161	Yash Sehrawat

ACKNOWLEDGEMENT

I express my sincere gratitude to VIT, Vellore, and the School of Computer Science and Engineering (SCOPE) for providing me with the opportunity and academic environment to undertake this project as part of BCSE406L – NoSQL Databases.

I sincerely thank my faculty guide, Dr. D. Vivek, for his valuable guidance, constructive suggestions, and continuous feedback throughout the project.

I also thank everyone who supported me in completing this project.

Finally, I thank everyone who directly or indirectly contributed to the successful completion of this project.

Team Member:
1. Yash Sehrawat
Team Name: SentinelStream
Batch ID: NS14

INDEX

S. No.	Contents	Page No.
i	Bonafide Certificate	i
ii	Acknowledgement	ii
iii	Index	iii
iv	Abstract	iv
1	**Chapter 1 – Introduction**	1
1.1	Background and Motivation	1
1.2	Problem Statement	1
1.3	Project Objectives	1
1.4	Scope of the Project	2
2	**Chapter 2 – Literature Survey**	3
2.1	Review of Existing Research	3
2.2	Research Gap	4
3	**Chapter 3 – Existing and Proposed System**	5
3.1	Existing System and Limitations	5
3.2	Proposed System	5
3.3	Features and Expected Outcomes	5
4	**Chapter 4 – Requirements and Technology Stack**	7
4.1	Hardware and Software Requirements	7
4.2	Functional and Non-functional Requirements	7
4.3	Technology Stack	7
5	**Chapter 5 – System Architecture and Design**	8
5.1	System Architecture	8
5.2	Module Design and Workflow	8
5.3	Database Design and Data Model	10
6	**Chapter 6 – NoSQL Database Implementation**	13
6.1	Database Configuration and Setup	13
6.2	Collections / Tables / Nodes / Keys	13
6.3	Implementation of Modules	14
6.4	Important Queries and Operations	15
7	**Chapter 7 – Testing and Results**	18
7.1	Test Cases and Test Results	18
7.2	Output Analysis	19
7.3	Discussion and Limitations	20
8	**Chapter 8 – Conclusion and Future Enhancements**	21
9	**References**	22
A1	**Annexure A – REVIEW 1 – PROPOSAL AND PLANNING**	24
A2	**Annexure B – REVIEW 2 – DESIGN AND PARTIAL IMPLEMENTATION**	25
A3	**Annexure C – REVIEW 3 – FINAL IMPLEMENTATION AND RESULTS**	26
A4	**Annexure D – Implementation Proof**	27
A5	**Annexure E – GitHub Repository and Code Submission Details**	35
A6	**Annexure F – Research paper draft or patent disclosure draft (Not Mandatory)**	36

ABSTRACT

Card fraud must be judged while a payment is in flight, yet organised fraud, where one device or network origin operates many stolen cards, is visible only in the connections between transactions. No single data model serves both needs. SentinelStream is a real-time fraud detection and fraud-ring discovery system built on polyglot NoSQL persistence. Its objectives are to score every transaction with explainable rules, keep bounded per-card state, retain alerts with varying evidence, resolve fraud rings by traversal, and present results to an analyst live. A Spring Boot service ingests about 150 synthetic transactions per second. Redis Streams provide ordered, redeliverable ingestion, and Redis sorted sets and GEO keys hold the sliding windows behind four deterministic rules: velocity, amount anomaly, impossible travel and entity fan-out. MongoDB is the system of record, with five validated collections, sixteen declared indexes, a complete CRUD triage workflow and five aggregation pipelines. Neo4j models cards, devices and IP addresses as a graph and resolves a ring in one bounded variable-length traversal when fan-out fires. A React dashboard shows transactions, alerts, rings and live query plans over WebSocket. In testing, the prototype sustained 150 transactions per second on one laptop with a mean scoring latency of about 17 ms, and indexes reduced documents examined by up to 3,914 times. Of 32 test cases, 31 passed; the one failure, a small loss of buffered transactions on a forced process kill, is analysed as a known limitation.

Keywords: NoSQL, polyglot persistence, Redis Streams, MongoDB, Neo4j, fraud detection, graph traversal, real-time systems

CHAPTER 1 – INTRODUCTION

1.1 Background and Motivation

Card payments are authorised in a few hundred milliseconds, so a fraud decision has to be made while the transaction is still in flight. A check that scans historical tables does not fit that budget. At the same time, the most damaging fraud is organised: a single compromised device or proxy IP address is used to operate many stolen cards, and each individual transaction looks unremarkable. Published work on fraud detection repeatedly identifies a continuous stream of transactions, class imbalance and changing behaviour as the defining conditions of the problem [1], [2], [6], and recent reviews continue to treat anomaly detection for financial fraud as an active research problem [15].

These two forms of fraud place opposite demands on the data layer. Behavioural checks need constant-time reads over recent activity. Ring discovery needs multi-hop traversal over a network of entities at a depth that is not known in advance. Investigators additionally need a durable record whose contents differ from one alert to the next. The motivation for this project is that NoSQL data models fit these needs well individually and that no one of them fits all three, which makes fraud detection a natural case for polyglot persistence: using different databases for different components of one system [18].

1.2 Problem Statement

Given a continuous stream of card transactions, design and implement a system that (a) scores every transaction against behavioural rules within the stream's arrival budget using only in-memory state; (b) maintains per-card state over sliding windows without unbounded memory growth; (c) detects coordinated fraud, in which cards are linked through shared devices or network origins and no single transaction is suspicious; (d) retains transaction and alert history for investigation, tolerating an alert payload whose shape varies with the rules that fired; and (e) presents all of this to an operator in real time.

1.3 Project Objectives

1. Ingest a continuous transaction stream through Redis Streams with ordered, at-least-once processing, idempotent writes and recovery when a consumer fails.
2. Score every transaction in real time with four explainable rules backed by bounded Redis sliding windows (sorted sets and GEO, every temporary key with a TTL).
3. Store transactions and alerts in MongoDB with schema validation, declared indexes, a full CRUD triage workflow and aggregation-pipeline analytics.

4. Model cards, devices and IP addresses in Neo4j and resolve fraud rings with one bounded variable-length Cypher traversal.
5. Present live transactions, alerts, rings and query plans to an analyst through a React dashboard over WebSocket, and measure throughput, memory and index effectiveness.

1.4 Scope of the Project

In scope: a synthetic transaction generator (about 150 transactions per second across 4,000 cards, 2,200 devices, 3,200 IP addresses, 80 merchants and 12 Indian cities); four injectable fraud patterns; the stream consumer and rules engine; three NoSQL stores on one machine; an analyst dashboard with triage, ring investigation and analytics; a reproducible seed script; and measured evidence.

Out of scope: real card data, machine-learning models, authentication, payment-network integration, and clustered or sharded deployment. Rules are deliberately deterministic so that any alert can be explained with exact numbers. Sharding is addressed as a design question (a hashed `cardId` shard key) and is not implemented.

CHAPTER 2 – LITERATURE SURVEY

2.1 Review of Existing Research

Fraud detection models. Bhattacharyya et al. compare random forests, support vector machines and logistic regression on real-life card data [5]. Dal Pozzolo et al. report practical lessons on imbalance, concept drift and performance assessment [1], and later formalise the operating conditions of real systems, in which supervised feedback arrives late and only for the few transactions investigators can check [2]. Surveys by Abdallah et al. [6], Hilal et al. [15] and Cherif et al. [16] map the field; Cherif et al. review 40 articles from 2015 to 2021 and call for more work on big-data analytics and large-scale deployment. Most of this research evaluates classifiers, and the data layer that must serve them in real time receives far less attention.

Streaming and real-time systems. Stonebraker et al. name fraud detection among the applications that strain traditional infrastructure and set out eight requirements for stream processing, including integrating stored and streaming data [13]. SCARFF combines Kafka, Spark and Cassandra with a learning approach for imbalance, non-stationarity and feedback latency [3]. Alarfaj and Shahzadi propose a graph neural network with a lambda architecture for real-time detection [17].

Graph-based detection. Akoglu et al. survey graph anomaly detection and stress attribution: being able to explain why something was flagged [7]. Pourhabibi et al. review graph-based fraud detection published from 2007 to 2018 and identify open challenges [14]. APATE combines customer spending features with network-based features, and reports that the combination gives its best models, with AUC above 0.98 [4]. This work establishes that connections carry evidence that individual transactions do not.

NoSQL stores and polyglot persistence. Cattell contrasts SQL and NoSQL stores on data model, consistency and durability [8]. Davoudian et al. explain the design decisions behind NoSQL stores [9], and Gessert et al. give decision guidance that maps requirements to techniques [10]. Angles and Gutierrez survey graph database models [11]. Khine and Wang review polyglot persistence and note that no single model fits all requirements of data-intensive systems [18]; Roy-Hubara et al. address how to select databases for such applications [19]; Kaur and Rani apply a polyglot framework to healthcare data [12]. Comparative benchmarks support the specific choices made here: Monteiro et al. found Neo4j best across their metrics on the LDBC Social Network Benchmark [20], and Carvalho et al. found MongoDB fastest of three document stores except on scan workloads [21].

Table 2.1: Comparison of the principal works reviewed

Ref.	Focus and method	Strength	Limitation for this problem
[5]	Random forest, SVM and logistic regression on real card data	Clear offline comparison of classifiers	Offline; each transaction judged alone, no shared-entity view
[1], [2]	Practitioner study and realistic formalisation of fraud detection	Models imbalance, drift and verification latency	Concerned with learning; storage and serving not addressed
[3]	SCARFF: Kafka, Spark and Cassandra with machine learning	Scalable, tested on a massive real dataset	Needs a cluster stack; no ring discovery
[4]	APATE: spending and network-based features	Shows network features add detection power	Features feed a classifier; rings are not surfaced to an analyst
[7], [14]	Surveys of graph-based anomaly detection	Structured view of methods, stress on explanation	Surveys; no real-time integrated system
[17]	Graph neural network with lambda architecture; autoencoder	Targets real-time detection, validated on bank case studies	Model output is harder to explain as exact numbers
[8], [9], [10]	Surveys and decision guidance for NoSQL stores	Principled basis for choosing a store	General; no fraud workload evaluated
[12], [18], [19]	Polyglot persistence: framework, review, selection	Justifies mixing data models	Not applied to a real-time fraud pipeline
[20], [21]	Benchmarks of graph and document databases	Measured evidence for Neo4j and MongoDB	Each store measured alone, not in combination

2.2 Research Gap

Across the papers reviewed, three gaps recur. First, detection research concentrates on model accuracy and says little about the data layer that has to answer in milliseconds. Second, graph-based work shows that connections reveal organised fraud, but it is generally offline or feeds a classifier; it does not hand off from a live behavioural signal to a traversal at the moment suspicion arises. Third, polyglot-persistence studies justify combining data models in general terms, without showing the combination working on a live fraud workload with measured results.

SentinelStream addresses this gap with four specific contributions: (1) a real-time handoff in which a Redis sliding-window signal triggers a Neo4j traversal that resolves the whole ring; (2) deterministic rules whose exact numeric evidence is stored in a polymorphic MongoDB document, so every flag is explainable [7]; (3) structural guarantees in the traffic generator that make false alerts impossible for three of the four rules by construction; and (4) index effectiveness shown live in the analyst's own interface.

CHAPTER 3 – EXISTING AND PROPOSED SYSTEM

3.1 Existing System and Limitations

Three approaches are common in the literature and in practice, each with limits for this problem.

Table 3.1: Existing approaches and their limitations

Approach	How it works	Limitations
Relational rules engine	Each check is a `COUNT(*)` or lookup against transaction tables	An index scan per authorisation; evidence of varying shape needs a sparse wide table or a key-value side table; ring discovery needs recursive self-joins of unknown depth
Offline machine-learning scoring	Classifiers trained and scored in batches [1], [5]	Delayed feedback [2]; each transaction judged alone; a score is harder to explain with exact numbers
Streaming framework with one store	Stream processor plus a single datastore [3]	Heavy cluster footprint; one data model serves windows, documents and connectivity alike; rings are not resolved

The honest counter-argument is that at low volume a single PostgreSQL instance with JSONB, recursive queries and materialised counters could do all of this. A polyglot design earns its complexity under volume and latency pressure and costs operational overhead in return. It is chosen here because the access patterns are genuinely different and because this course is concerned with matching data models to them.

3.2 Proposed System

SentinelStream assigns each access pattern to the store whose data model makes it native.

Table 3.2: Why each NoSQL database was selected

Requirement	Store	Why this model, and why not the others
Ordered, replayable ingestion with redelivery	Redis Streams [22]	An append-only log with per-consumer acknowledgement and a pending list. A relational queue needs polling and locking; plain Pub/Sub loses messages when the consumer is down.
Sliding-window counts, rolling statistics, distance	Redis sorted sets and GEO [23]	The score is the timestamp, so a window is trimmed by score and counted in constant time. The SQL equivalent is an indexed `COUNT(*)` on every transaction.
Alerts with varying evidence; history; triage; analytics	MongoDB	Each alert holds an array of rule objects whose fields differ by rule. Validators, TTL indexes and aggregation pipelines suit a system of record.
Multi-hop ring discovery	Neo4j	A ring is a connected group of unknown depth, expressed as one variable-length pattern. In SQL it needs a recursive self-join whose cost grows with depth.

3.3 Features and Expected Outcomes

- **Live scoring** of every transaction, with score = min(100, sum of rule weights) and bands Low 1–39, Medium 40–69, High 70–100.
- **Four injectable fraud patterns**, so each rule can be demonstrated on demand.
- **Automatic ring discovery:** fan-out in Redis triggers a Neo4j traversal, saved as a snapshot and pushed to the dashboard.

- **Full triage workflow:** raise, read, assign, annotate, resolve and dismiss alerts, with an audit trail.
- **Cross-store card blocking:** a MongoDB transaction, a Redis blocklist entry and a graph flag, after which the card's next payment is declined.
- **Analytics** from five aggregation pipelines, and **live query plans** in the triage view.

Expected outcomes were: sustained throughput of about 150 transactions per second on a laptop; scoring in milliseconds; close to no false alerts from normal traffic; visible index effectiveness; and one-command startup. Chapter 7 reports the measured results against these.

CHAPTER 4 – REQUIREMENTS AND TECHNOLOGY STACK

4.1 Hardware and Software Requirements

Table 4.1: Hardware and software used

Item	Specification
Processor	Intel Core i5-12450H (12 logical processors), as tested
Memory	7.7 GB RAM, as tested; the stack used up to about 1.25 GB
Disk	About 2 GB (1.5 GB of portable tools, the rest data)
Operating system	Windows 11; no administrator rights and no Docker needed
Runtimes	Temurin JDK 21, Node.js 20.14 or newer
Databases	Redis 7.2.16, MongoDB 7.0.14 with mongosh 2.3.1, Neo4j 5.20.0 Community

4.2 Functional and Non-functional Requirements

Table 4.2: Functional requirements

ID	Requirement
FR-1	Generate synthetic transactions continuously and allow traffic to be paused and resumed.
FR-2	Ingest through a stream and process each transaction in the fixed ten-step order, acknowledging last.
FR-3	Evaluate velocity, amount-anomaly, impossible-travel and entity fan-out rules, storing exact evidence.
FR-4	Provide on-demand injection of the four fraud patterns.
FR-5	Provide create, read, update and delete on alerts, enforcing the status lifecycle on the server.
FR-6	Block and unblock a card across stores, declining a blocked card's later payments.
FR-7	Resolve a fraud ring for any card and sweep the graph for heavily shared entities.
FR-8	Show aggregated statistics, live query plans and the index registry in the dashboard.

Table 4.3: Non-functional requirements

ID	Attribute	Requirement
NFR-1	Performance	About 150 transactions/s, scored in milliseconds; nothing blocking on the hot path.
NFR-2	Reliability	At-least-once delivery; no duplicate effects on redelivery; orphans reclaimed.
NFR-3	Bounded memory	TTL on every temporary Redis key; stream capped; transactions expire in 24 h.
NFR-4	Explainability	Rules are deterministic; each alert carries the numbers that triggered it.
NFR-5	Query efficiency	Every MongoDB query on a request path is backed by a declared index.
NFR-6	Operability	One command starts everything on freshly seeded data; one command stops it.
NFR-7	Data integrity	JSON Schema validators on all collections; uniqueness constraints in Neo4j.

4.3 Technology Stack

Table 4.4: Technology stack

Layer	Technology	Purpose
Front end	React 18, Vite 5, TanStack Query, Recharts, Cytoscape, STOMP.js	Dashboard, charts, ring graph
Back end	Java 21, Spring Boot 3.3 [28], Redis, MongoDB and Neo4j drivers	Consumer, detectors, REST, STOMP
NoSQL stores	Redis 7.2, MongoDB 7.0 (replica set `rs0`), Neo4j 5.20	Windows; records; entity graph
Build and scripts	Gradle 8.8 wrapper, npm, Node.js, PowerShell	Build, seed and evidence scripts
Deployment	Local native processes on 127.0.0.1; no cloud, no Docker	Fits a 7.7 GB laptop
Other tools	Git, GitHub, Neo4j Browser, mongosh, redis-cli, headless Edge	Versioning, inspection, UI checks

CHAPTER 5 – SYSTEM ARCHITECTURE AND DESIGN

5.1 System Architecture

Five processes run on one machine: Redis (port 6379), MongoDB (27017), Neo4j (7687, with Browser on 7474), the Spring Boot API (8080) and the React dashboard (3000). Figure 5.1 shows them and labels each connection with the command that carries it. The generator writes to a Redis Stream; the consumer reads it through a consumer group, updates sliding windows, runs the detectors, and writes to MongoDB and Neo4j through buffers; the API serves the dashboard over REST and STOMP.

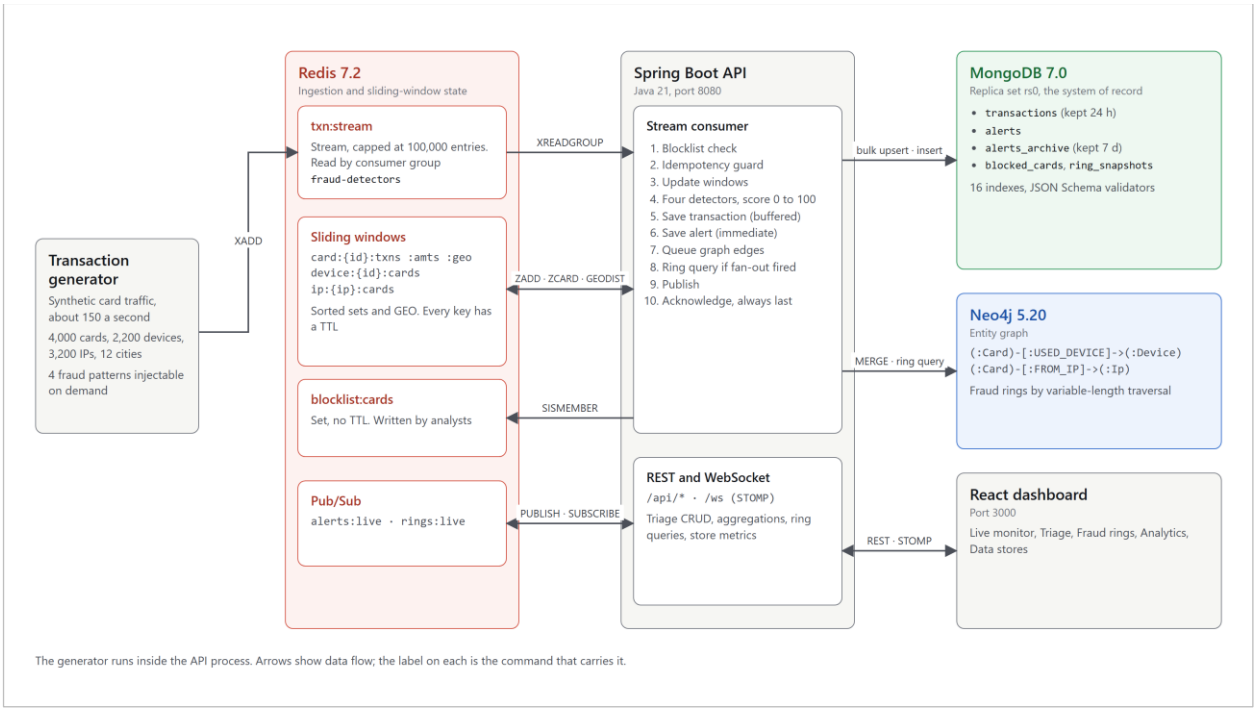


Figure 5.1: System architecture

Four principles shaped the design. **Nothing blocking on the hot path:** no Neo4j write, no per-transaction MongoDB insert and no ring query runs inline. **Acknowledge last:** `XACK` is always the final step, so a crash before it means redelivery. **Deterministic rules:** there is no machine learning, and an alert can always be explained with exact numbers. **Bounded by design:** memory is limited by TTLs and caps, not by eviction.

5.2 Module Design and Workflow

Table 5.1: Modules (Java packages under com.sentinelstream)

Module	Responsibility
`generator`	Population, per-card spending profile and cooldown; writes `txn:stream` with `XADD`
`consumer`	Consumer group `fraud-detectors`; the ten-step pipeline; reclaims orphaned entries
`detect`	Sliding-window updates, the four detectors, and the rules engine (score and band)
`persist`	Buffered bulk writer for transactions, synchronous alert writer, ring snapshot writer

Module	Responsibility
`graph`	Batched projector into Neo4j, the two ring queries, the fan-out trigger with cooldown
`triage`	Alert CRUD, status lifecycle, and the cross-store block-card service
`stats, admin`	Aggregation pipelines A to D; index registry and store metrics
`control, ws`	Fraud injectors and generator control; STOMP broker, Redis relay and sampled tape

Workflow. Each transaction passes through ten steps in a fixed order (Figure 5.2). The order is a correctness requirement: blocklist first, because a blocked card should not update behavioural windows; idempotency before any mutation; windows before detectors, because detectors read them. Transactions are buffered (200 documents or 500 ms) while alerts are written synchronously, because alerts are rare, are the durable record, and must be readable the moment the dashboard is told about them.

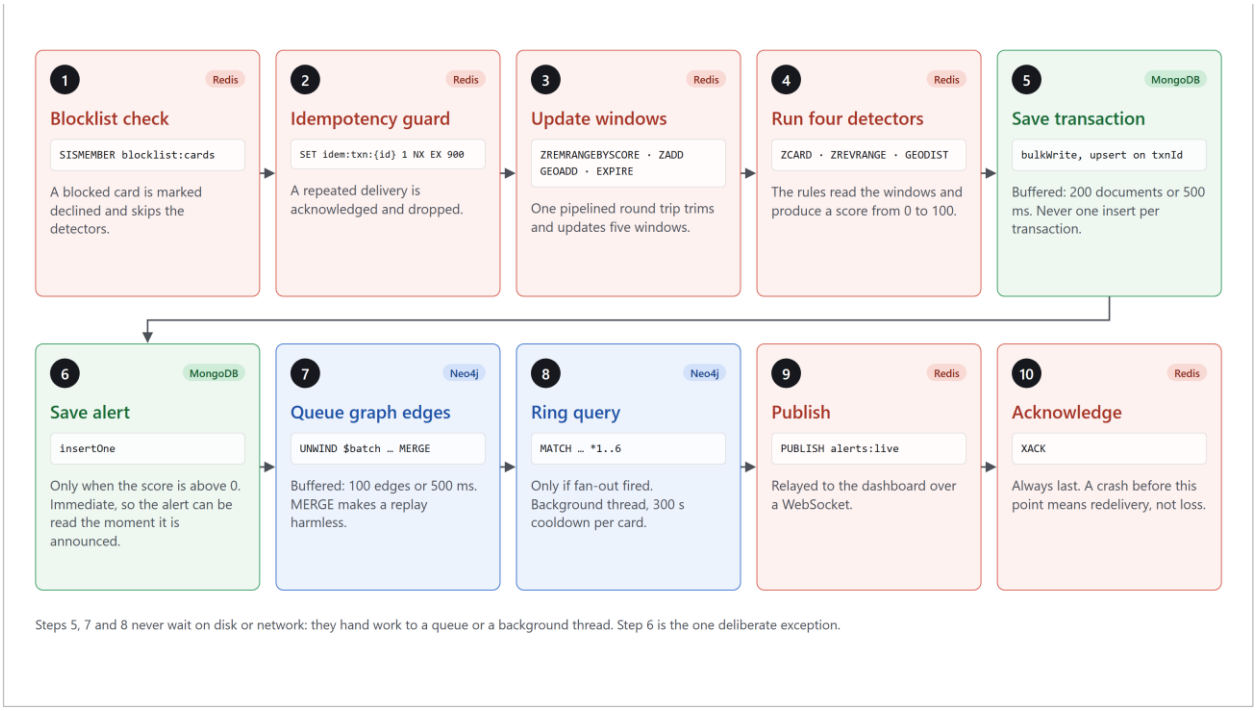


Figure 5.2: The ten-step consumer pipeline, coloured by store

Table 5.2: Detection rules (score = min(100, sum of points))

Rule	Fires when	Redis read	Points
Velocity	5 or more transactions on one card in 60 s	`ZCARD card:{id}:txns`	40
Amount anomaly	Amount above mean + 3σ of the card's last 20 amounts (at least 5)	`ZREVRANGE card:{id}:amts`	25
Impossible travel	Implied speed above 900 km/h across at least 200 km	`GEODIST card:{id}:geo`	70
Entity fan-out	One device or IP used by 3 or more cards in 600 s	`ZCARD device:{id}:cards`	55

Keeping normal traffic quiet. The generator gives each rule a structural guarantee rather than a statistical hope. A card waits at least 20 s between payments, so at most 4 fall in any 60 s (limit 5). Each device and IP is shared by at most 2 cards (limit 3). Each card keeps one home city. Each card

stays within ±40% of its own typical spend, capped at mean + 2.5σ of its history; about 1 payment in 3,000 is a deliberate overspend, so the amount rule stays visibly active.

The Redis to Neo4j handoff (Figure 5.3) is the point of the three-store design. Redis notices that a device has reached three cards, but cannot say who else is connected. That triggers one Neo4j traversal, run on a background thread one second later so the triggering edge has been written. The result is saved in MongoDB and pushed to the analyst.

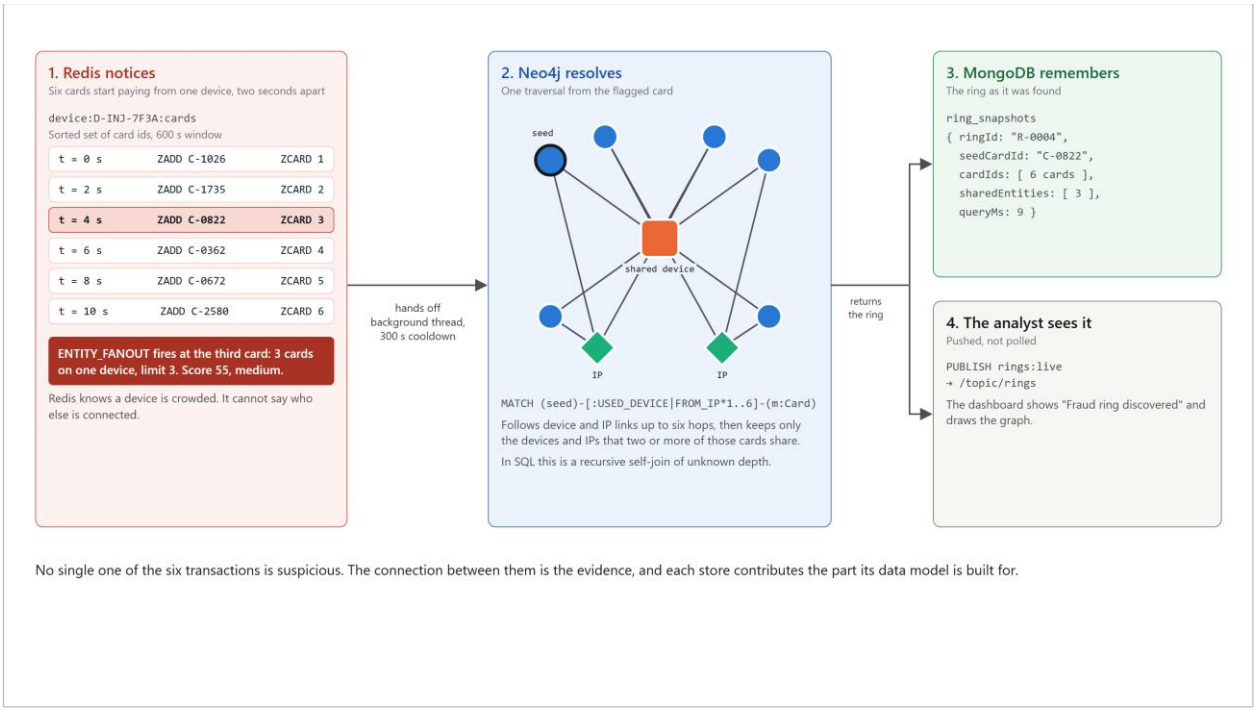


Figure 5.3: Redis notices, Neo4j resolves, MongoDB remembers, the analyst sees it (card and device ids illustrative)

5.3 Database Design and Data Model

Figure 5.4 shows the three data models side by side. They were designed from the queries each store must answer, not derived from a relational schema.

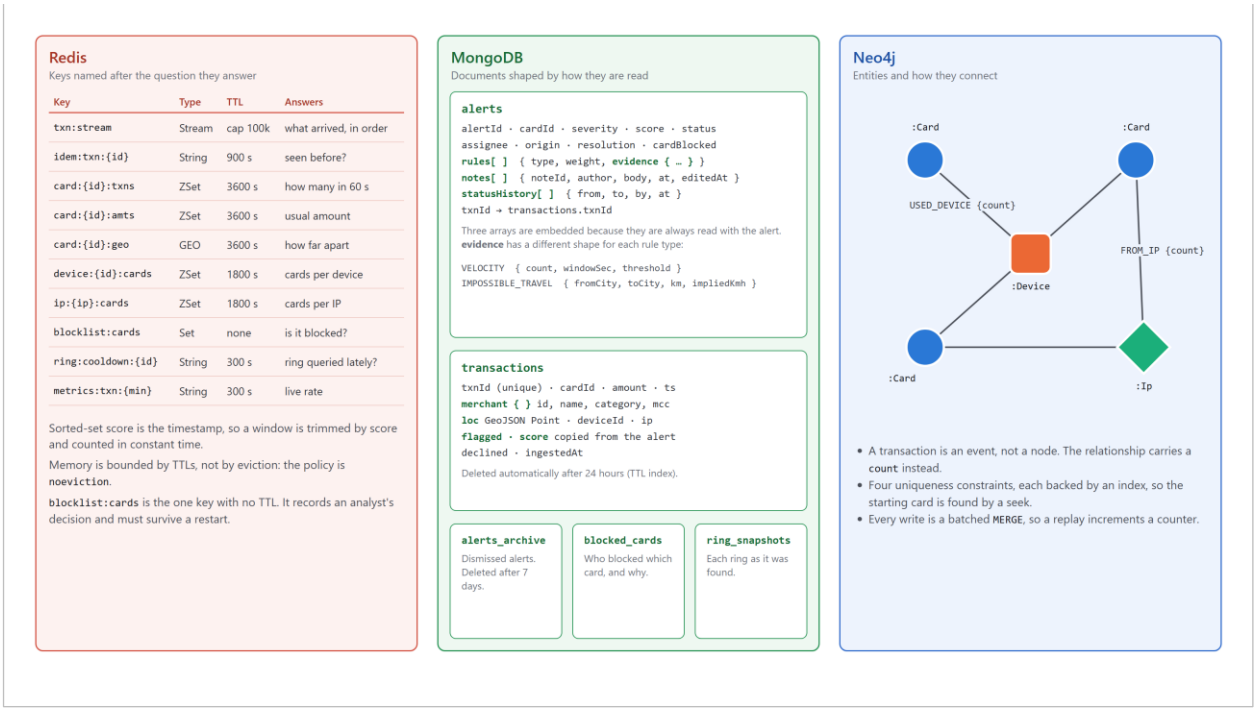


Figure 5.4: Data model of each store

Table 5.3: Document-model design decisions in MongoDB

Decision	What was done	Why
Embed	`rules[]`, `notes[]` and `statusHistory[]` live inside the alert	They belong to one alert and are always read with it
Polymorphic sub-document	`rules[].evidence` has no fixed shape; the validator allows any object	Each rule proves something different
Denormalise for reads	`flagged` and `score` are copied onto each transaction	Pipeline A needs no join on every refresh
Bounded arrays	Notes use `$push` with `$slice: -50`	Avoids unbounded document growth
Reference	`alerts.txnId` points to `transactions.txnId`	Lifetimes differ: 24 hours against indefinite
Hot and cold split	TTL index on transactions; alerts keep their own evidence	An expired transaction never orphans an alert

In Neo4j a transaction is an event, not an entity. At 150 per second, modelling transactions as nodes would add about 13 million nodes a day that nothing traverses, so each relationship carries a `count` instead. Alert status follows a server-enforced lifecycle, and blocking a card touches all three stores in a defined order (Figure 5.5). MongoDB runs as a single-node replica set because multi-document transactions require one [26].

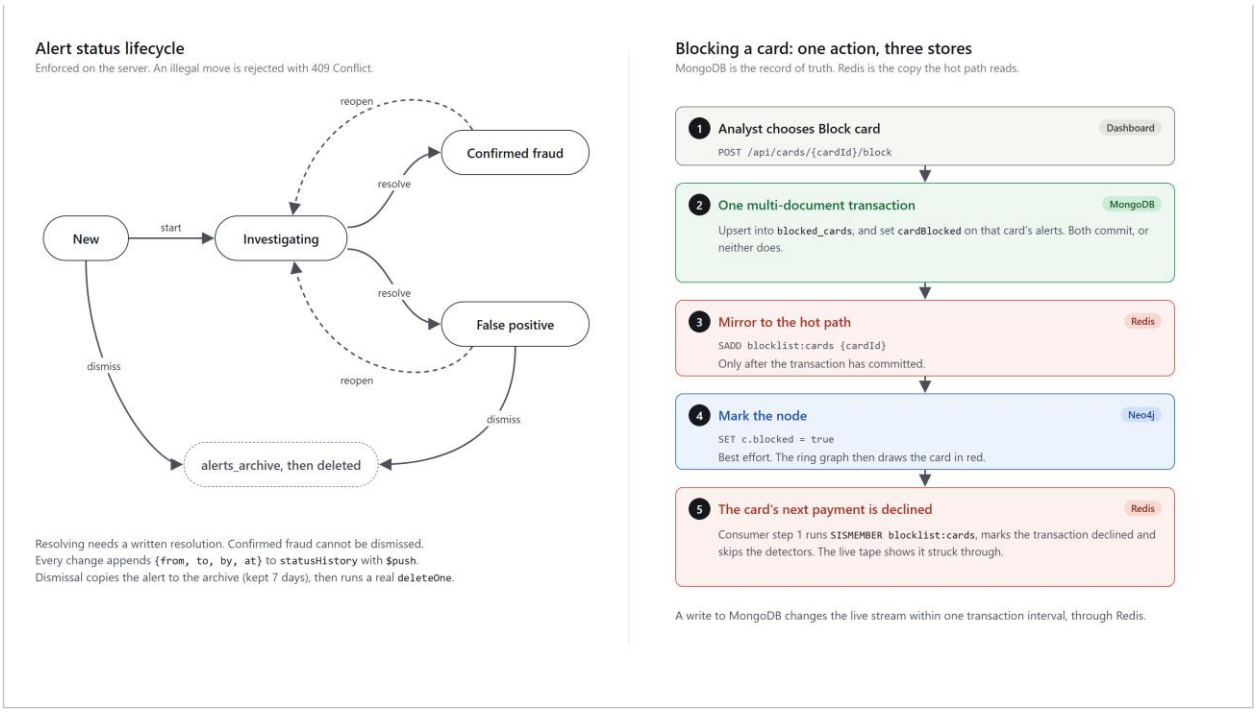


Figure 5.5: Alert status lifecycle and the cross-store block-card flow

CHAPTER 6 – NOSQL DATABASE IMPLEMENTATION

6.1 Database Configuration and Setup

`scripts/native/setup-native.ps1` unpacks portable builds, verifying SHA-256 checksums where the publisher provides them. `start-native.ps1` then starts each store with the settings in Table 6.1, tuned for a 7.7 GB laptop.

Table 6.1: Store configuration

Store	Key settings	Reason
Redis	`appendonly yes`, `appendfsync everysec`, `save ""`, `maxmemory 256mb`, `maxmemory-policy noeviction`	At most about one second lost on a crash. `allkeys-lru` could silently evict the blocklist or pending list, so memory is bounded by TTLs and writes fail loudly under pressure.
MongoDB	`--replSet rs0`, `--wiredTigerCacheSizeGB 0.25`, `--bind_ip 127.0.0.1`	A replica set enables multi-document transactions.
Neo4j	Heap 256–384 MB, page cache 128 MB	Lowered, never raised, to fit the laptop.
API	`-Xmx384m`; pooled Redis connections (maximum 16)	Without the pool, one TCP connection per transaction exhausted the operating system's ports.

Indexes, validators and constraints are created idempotently by code at startup (`MongoIndexConfig`, `MongoValidatorConfig`, `Neo4jConstraintsConfig`), so they are declared in source and never made by hand. Validators are applied with `collMod` at every boot, at `validationLevel: moderate`, `validationAction: error`. An excerpt:

```
"severity": { "enum": ["LOW","MEDIUM","HIGH"] },
"origin":   { "enum": ["DETECTOR","ANALYST"] },
"txnId":    { "bsonType": ["string","null"] },
"rules": { "bsonType": "array", "items": { "bsonType": "object",
    "required": ["type","weight","evidence"],
    "properties": { "type":     { "bsonType": "string" },
                    "weight":   { "bsonType": ["int","double"] },
                    "evidence": { "bsonType": "object" } } } }
```

6.2 Collections / Tables / Nodes / Keys

MongoDB: database sentinel, five collections

Table 6.2: MongoDB collections

Collection	One document is	Retention
`transactions`	One card payment	24 hours (TTL index)
`alerts`	One alert with its whole case history	Indefinite
`alerts_archive`	A dismissed alert, with who, when and why	7 days (TTL index)
`blocked_cards`	One block: who, when and why	Indefinite
`ring_snapshots`	One ring as it was found	Indefinite

A real alert document, read from the running system after an analyst moved it to Investigating, added a note and assigned it:

```
{ "alertId": "A-9000024", "txnId": "T-MV6XBKEC-00063412", "cardId": "C-0170",
  "severity": "HIGH", "score": 70, "origin": "DETECTOR",
  "rules": [ { "type": "IMPOSSIBLE_TRAVEL", "weight": 70,
      "evidence": { "fromCity": "Chennai", "toCity": "Delhi", "km": 1756.3,
                    "seconds": 2, "impliedKmh": 3125385.1, "threshold": 900 } } ],
  "status": "INVESTIGATING", "assignee": "analyst.dev",
  "notes": [ { "noteId": "n-7b6f292c", "author": "analyst.yash",
               "body": "Called the cardholder: they are in Chennai and did not travel." } ],
  "statusHistory": [ { "from": null,  "to": "NEW",           "by": "system" },
                     { "from": "NEW", "to": "INVESTIGATING", "by": "analyst.yash" } ],
  "cardBlocked": false, "resolution": null }
```

The evidence shapes of the other rules, from real alerts, show why alerts are documents:

```
VELOCITY       { "count": 15, "windowSec": 60, "threshold": 5 }
AMOUNT_ANOMALY { "amount": 7046.61, "mean": 2264.12, "stddev": 449.69, "sigma": 3,
                 "threshold": 3613.19, "sampleSize": 20 }
ENTITY_FANOUT  { "windowSec": 600, "threshold": 3, "deviceId": "D-INJ-CF5A332E",
                 "deviceCardCount": 6, "ip": "10.20.228.125", "ipCardCount": 3 }
```

Redis: keys designed per access pattern

Table 6.3: Redis key registry

Key	Type	TTL	Answers
`txn:stream`	Stream	cap 100,000	What arrived, in order
`idem:txn:{id}`	String	900 s	Has this been processed?
`card:{id}:txns`	Sorted set	3600 s	How many in the last 60 s?
`card:{id}:amts`	Sorted set	3600 s	What is this card's usual amount?
`card:{id}:geo`	GEO	3600 s	How far apart were the last two payments?
`device:{id}:cards`, `ip:{ip}:cards`	Sorted set	1800 s	How many cards on this device or IP?
`blocklist:cards`	Set	none	Is this card blocked? Survives restarts by design
`ring:cooldown:{id}`	String	300 s	Was a ring queried for it lately?
`metrics:txn:{minute}`	String	300 s	Live ingest rate
`alerts:live`, `rings:live`	Pub/Sub	none	Live notifications to the dashboard

Neo4j: nodes and relationships

Nodes: `Card {cardId}`, `Device {deviceId}`, `Ip {ip}`. Relationships: `(:Card)-[:USED_DEVICE {count}]->(:Device)` and `(:Card)-[:FROM_IP {count}]->(:Ip)`. Four uniqueness constraints each create a backing index. The schema also defines a `Merchant` node and `PAID` relationship, which the live system does not write, because ring detection needs only devices and IP addresses.

6.3 Implementation of Modules

Stream consumer. `TransactionConsumer.process` applies the ten steps in order. Failure leaves the entry unacknowledged in the pending list, and a scheduled task reclaims entries that stay idle for over a minute.

```
boolean blocked = redis.opsForSet().isMember(BLOCKLIST_KEY, cardId);            // 1 blocklist
Boolean acquired = redis.opsForValue().setIfAbsent(IDEM_KEY_PREFIX + txnId, "1",
        Duration.ofSeconds(keysProps.getIdempotencyTtl()));                     // 2 idempotency
if (!Boolean.TRUE.equals(acquired)) { ack(record); return; }
if (!blocked) { windows.updateWindows(...);                                     // 3 windows
                eval = rulesEngine.evaluate(...); }                             // 4 detectors
bulkWriter.enqueue(mapper.toDocument(v, blocked, flagged, score));              // 5 buffered
alertDoc = alertWriter.write(txnId, cardId, ts, eval);                          // 6 if score > 0
graphProjector.enqueue(cardId, deviceId, ip, ts);                               // 7 buffered
ringQueryTrigger.maybeTrigger(cardId);                                          // 8 if fan-out fired
publishAlert(alertDoc);  txnBroadcaster.maybePublish(v, flagged, blocked);      // 9 publish
ack(record);                                                                    // 10 XACK last
```

Sliding windows and detectors. `WindowService` sends each trim, add and expire in one pipelined round trip. Because adding the same card twice only moves its score, `ZCARD` on a device's set is already a count of distinct cards.

Bulk writer. `MongoBulkWriter` uses a bounded queue and an unordered `bulkWrite` of `ReplaceOneModel` upserts keyed on `txnId`, so repeating a write is harmless.

Graph projector. Edges are batched and written with `UNWIND` and `MERGE`, never `CREATE`, so a replay only increments a counter:

```
UNWIND $batch AS row
MERGE (c:Card {cardId: row.cardId}) ON CREATE SET c.firstSeen = row.ts, c.txnCount = 0
SET c.lastSeen = row.ts, c.txnCount = c.txnCount + 1
MERGE (d:Device {deviceId: row.deviceId}) ON CREATE SET d.firstSeen = row.ts
MERGE (c)-[ud:USED_DEVICE]->(d) ON CREATE SET ud.count = 0, ud.firstSeen = row.ts
SET ud.count = ud.count + 1, ud.lastSeen = row.ts          // FROM_IP is written the same way
```

Cross-store block. `CardBlockService` commits to MongoDB first, because it is the record of truth, and only then updates the Redis copy that the hot path reads:

```
session.withTransaction(() -> {
    blockedCards.replaceOne(session, Filters.eq("cardId", cardId), blockDoc, upsert);
    alerts.updateMany(session, Filters.and(Filters.eq("cardId", cardId),
        Filters.ne("status", "FALSE_POSITIVE")), Updates.set("cardBlocked", true));
    return null; });
redis.opsForSet().add(BLOCKLIST_KEY, cardId);    // only after the commit
markInGraph(cardId, true);                       // best effort: the graph is derived
```

Dashboard. The React client has five views: Live monitor, Triage, Fraud rings, Analytics and Data stores. It receives `/topic/txn` (sampled to about 8 rows a second, with flagged and declined rows always sent), `/topic/alerts` and `/topic/rings` over STOMP, and polls the KPI pipeline every 3 s. Annexure D shows each view.

6.4 Important Queries and Operations

CRUD operations

Table 6.4: CRUD on the alerts collection

Operation	REST endpoint	MongoDB call
Create	`POST /api/alerts/manual`	`insertOne, origin: "ANALYST", txnId: null`

Operation	REST endpoint	MongoDB call
Read	`GET /api/alerts?status=&severity=`, `GET /api/alerts/{id}`	`find` with filter, sort and paging; aggregation with `$lookup`
Update	`PATCH …/status`, `…/assign`, `POST`, `PATCH`, `DELETE …/notes`	`findOneAndUpdate` with `$set`, `$push` + `$slice`, `$pull`, `arrayFilters`
Delete	`DELETE /api/alerts/{id}`, `DELETE /api/alerts?status=`	Copy to `alerts_archive`, then `deleteOne` / `deleteMany`

Indexing

Sixteen indexes are declared in code: unique (#1, #7, #14), compound (#2, #4, #8, #9, #10), single-field (#3, #11, #15, #16), multikey (#12), TTL (#5, #13) and geospatial (#6). Figure D.14 lists them from the live database. Index #10, `{status: 1, severity: 1, ts: -1}`, **follows the ESR guideline** [24]: equality fields first, then the sort field. MongoDB seeks straight to the matching block, which is already newest first; a leading `ts` would turn the equality tests into a filter over the whole time range (Figure 6.1).

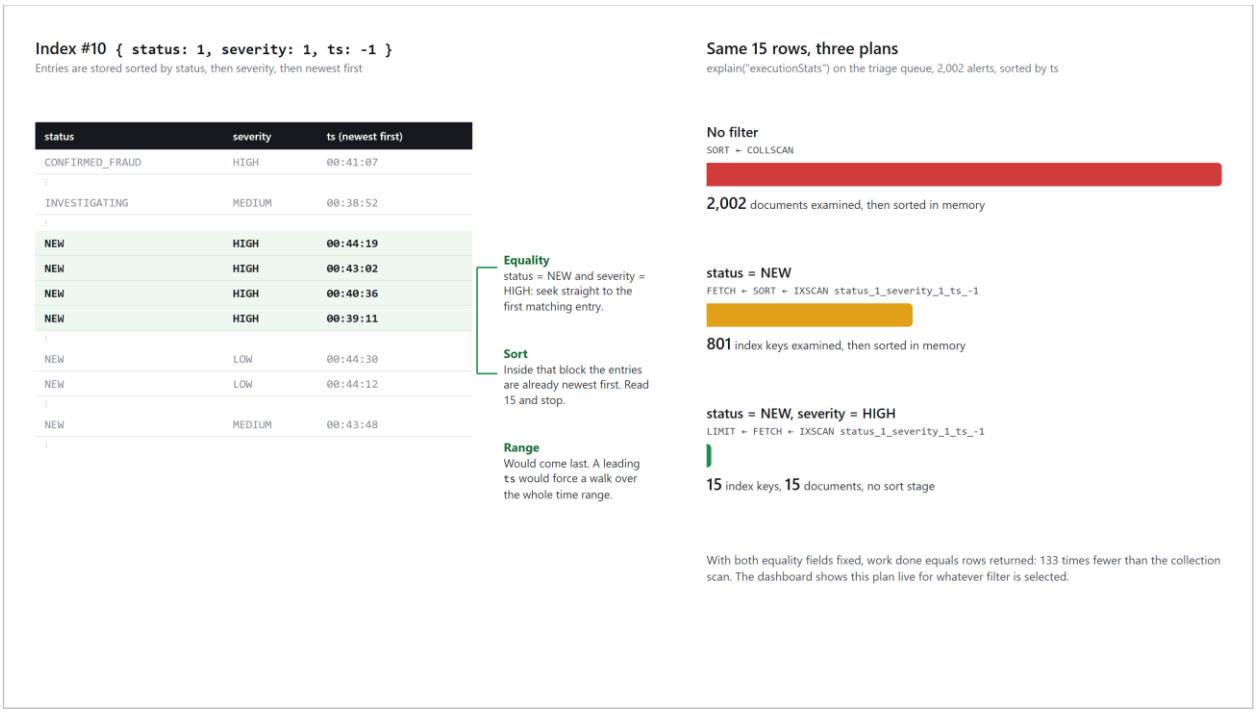


Figure 6.1: Index #10 and three measured plans for the same 15 rows

Aggregation pipelines

Table 6.5: Five aggregation pipelines

	Drives	Key stages and operators
A	Fraud rate by merchant category	`$match`, `$group` with `$cond`, `$match` on group size, `$addFields`, `$sort`, `$limit`
B	Alerts by severity over time	`$group` with `$dateTrunc`, `$group`, `$mergeObjects`, `$arrayToObject` (empty severities filled with zero)
C	Watchlist of top cards	`$group` with `$addToSet`, `$reduce` with `$setUnion` to flatten arrays of arrays
D	KPI tiles, detector activity, score bands	`$facet` with `$count`, `$group`, `$unwind`, `$bucket`: seven statistics in one round trip

	Drives	Key stages and operators
E	Alert detail drawer	Three `$lookup` stages, one correlated: alert, its transaction, 10 prior transactions, block status

Pipeline A, and the correlated join in pipeline E, which index #2 serves [25]:

```
db.transactions.aggregate([
  { $match: { ts: { $gte: since } } },
  { $group: { _id: "$merchant.category", total: { $sum: 1 },
              flagged: { $sum: { $cond: ["$flagged", 1, 0] } }, volume: { $sum: "$amount" } } },
  { $match: { total: { $gte: 20 } } },
  { $addFields: { fraudRate: { $round: [{ $divide: ["$flagged", "$total"] }, 4] } } },
  { $sort: { fraudRate: -1 } }, { $limit: 12 } ])

{ $lookup: { from: "transactions", let: { card: "$cardId", at: "$ts" }, pipeline: [
    { $match: { $expr: { $and: [ { $eq: ["$cardId", "$$card"] }, { $lt: ["$ts", "$$at"] } ] } } },
    { $sort: { ts: -1 } }, { $limit: 10 } ], as: "priorTxns" } }
```

The ring query

`RING_FOR_CARD` [27] follows device and IP links up to six hops, then keeps only entities shared by at least two ring members, so the result shows shared infrastructure rather than every device each card ever used. The bound is deliberate: an unbounded match over a connected payment graph tends towards scanning all of it, and cards six hops apart are barely related. A second query, `ALL_RINGS`, ranks entities used by three or more cards.

```
MATCH (seed:Card {cardId: $cardId})
CALL { WITH seed
       MATCH (seed)-[:USED_DEVICE|FROM_IP*1..6]-(m:Card) WHERE m <> seed
       RETURN collect(DISTINCT m) AS others }
WITH [seed] + others AS cards
UNWIND cards AS c
MATCH (c)-[r:USED_DEVICE|FROM_IP]->(e)
WITH cards, c, r, e
WHERE size([ (x:Card)-[:USED_DEVICE|FROM_IP]->(e) WHERE x IN cards | x ]) >= 2
RETURN [x IN cards | { id: x.cardId, blocked: coalesce(x.blocked, false) }] AS cardNodes,
       collect(DISTINCT { id: coalesce(e.deviceId, e.ip), type: head(labels(e)) }) AS entityNodes,
       collect(DISTINCT { from: c.cardId, to: coalesce(e.deviceId, e.ip),
                          rel: type(r), weight: r.count }) AS edges
```

CHAPTER 7 – TESTING AND RESULTS

7.1 Test Cases and Test Results

All tests were run on 3 November 2026 against the live system, on the machine in Table 4.1, using scripts that read each actual result from the system itself. `scripts/smoke.ps1` repeats the core path in seconds (Figure D.19).

Test Case ID	Feature / Input	Expected Result	Actual Result	Status
TC-01	System health: `GET /api/health`	All three stores report UP	HTTP 200; Redis, MongoDB and Neo4j UP	Pass
TC-02	Throughput over 60 s of normal traffic	About 150 transactions/s	9,015 in 60.0 s = 150.2/s	Pass
TC-03	Scoring latency in steady state (`ingestedAt − ts`, 3-minute window, 26,250 transactions)	Milliseconds; no consumer lag	Mean 17 ms, p95 29 ms, p99 36 ms, maximum 64 ms; lag 0	Pass
TC-04	False alerts: 44,439 live transactions, no injection	No velocity, travel or fan-out; amount rule about 1 in 3,000	14 amount alerts (0.03%); 0 velocity, 0 travel, 0 fan-out	Pass
TC-05	Inject velocity burst	Medium, score 40, from the 5th transaction	11 alerts; first Medium, 40, count 5 of limit 5 in 60 s	Pass
TC-06	Inject unusual amount	Low, score 25, amount above threshold	Low, 25; ₹17,328.09 above ₹7,236.90 (mean 4,277.71, 20 samples)	Pass
TC-07	Inject impossible travel, wait 70 s	One alert only; High, score 70	1 alert; High, 70; Chennai to Delhi, 1,756.3 km in 2 s	Pass
TC-08	Inject fraud ring (6 cards)	Fan-out from the 3rd card; ring resolved	Alerts per card 0, 0, 1, 1, 1, 1; 4 snapshots show the device with 3, 4, 5, 6 cards; 6 of 6 cards found; 7–12 ms	Pass
TC-09	Seeded ring: `GET /api/rings/C-9001`	6 cards, 3 shared entities, C-9004 blocked	As expected, 5 ms	Pass
TC-10	Create: raise a manual case	Count +1; origin ANALYST; null txnId	Count 2,096 to 2,097; origin ANALYST; txnId null	Pass
TC-11	Read: queue New + High; alert detail	Filtered, newest first; joined data returned	15 of 87 rows, correctly filtered and sorted; transaction joined	Pass
TC-12	Update: NEW to INVESTIGATING	Status changes; audit entry added	HTTP 200; `statusHistory` length 1 to 2	Pass
TC-13	Illegal move: NEW to CONFIRMED_FRAUD	HTTP 409; status unchanged	HTTP 409, “Illegal transition”; status stays NEW	Pass
TC-14	Resolve without a written resolution	HTTP 400; accepted once supplied	HTTP 400, then HTTP 200	Pass
TC-15	Notes: add, edit, delete	`$push`, `arrayFilters`, `$pull` work	Note added, edited with `editedAt` set, removed	Pass
TC-16	Assign an alert	Assignee stored	HTTP 200; assignee analyst.dev	Pass
TC-17	Delete: dismiss one alert	alerts −1; archive +1 with reason	alerts 2,102 to 2,101; archive 4 to 5; reason kept	Pass
TC-18	Dismiss a confirmed-fraud alert	Refused	HTTP 403; alert kept	Pass
TC-19	Bulk dismiss false positives	`deleteMany` count equals matches	deletedCount 300; alerts 2,101 to 1,801; archive 5 to 305	Pass
TC-20	Invalid documents (bad severity; text amount)	Rejected by validators	Both rejected, code 121, Document failed validation	Pass
TC-21	Polymorphic evidence in one collection	Different fields per rule accepted	Four rules stored four different field sets	Pass
TC-22	Index registry and constraints	16 indexes, 2 TTLs, 4 constraints	16 plus 5 `_id`; TTL 86,400 s and 604,800 s; 4 constraints	Pass
TC-23	Query plan: New + High, limit 15	IXSCAN on index #10; no sort; keys = returned	`status_1_severity_1_ts_-1`, 15 keys, 15 documents, no sort	Pass
TC-24	Pipelines A to E	Correct shapes; agree with direct counts	KPI 1,236 equals direct count 1,236; JEWELRY ranks first; 13 buckets; 8 cards; join returned	Pass
TC-25	Block a card, then unblock it	Written to all three stores; later payment declined; reversed	Active in MongoDB, SISMEMBER 1, Neo4j true; 1 payment declined; after unblock 0 and false	Pass
TC-26	Idempotency: same stream entry written twice	One document; no extra alert	1 document; alerts unchanged	Pass

Test Case ID	Feature / Input	Expected Result	Actual Result	Status
TC-27	MongoDB TTL: insert with `ingestedAt` 48 h old	Removed by the TTL monitor	Removed about 5 s after insert	Pass
TC-28	Redis TTL: sample 2,000 keys	All have a TTL except the stream and blocklist	0 sampled keys without a TTL	Pass
TC-29	Crash recovery: force-kill API, restart	Pending entries reclaimed; no duplicates	9 pending at kill, 0 within 60 s of restart; 0 duplicate `txnId`	Pass
TC-30	Crash durability: same kill, count documents	Every generated transaction persisted	33 of 176,157 (0.019%) missing: acknowledged but still in the write buffer	Fail
TC-31	Data wipe while the API runs	Consumer group recreated; processing continues	Group recreated; lag 0; 2,250 more generated and 2,235 stored in 15 s; 1 warning logged	Pass
TC-32	Dashboard in headless Edge: all five views and workflows	No script errors or failed requests	No console errors, exceptions or failed requests	Pass

Summary: 31 of 32 test cases passed; TC-30 failed and is analysed in Section 7.3.

7.2 Output Analysis

Table 7.1: Measured performance

Measure	Result
Sustained throughput	150.2 transactions/s (target 150)
Scoring latency, steady state	Mean 17 ms, p95 29 ms, p99 36 ms, maximum 64 ms (26,250 transactions)
Scoring latency, whole run including startup	Median 38 ms, p95 199 ms, p99 4.4 s, maximum 6.1 s: the backlog while the JVM warms up
Write buffers	0 dropped by the bulk writer (44,963 documents) and the projector (44,966 edges)
False-alert rate	14 of 44,439 (0.03%), all from the deliberate overspend trickle
Ring query	3–12 ms once warm, on a graph of about 9,100 nodes
Memory, whole stack (working set)	About 1.25 GB at the highest snapshot (Neo4j about 540 MB, MongoDB 329, API 263, Redis 84, UI 27); 810 MB in a later snapshot

Table 7.2: Index evidence: each target index dropped, measured, recreated and measured again

Probe	Query	Without the index	With the index	Improvement
P1	Queue: New + High, by time	Fallback index #8, 172 docs	Index #10, 87 docs for 87 returned	2× fewer
P2	Card history, limit 10	`ts` index, 38,895 docs, 51 ms	Index #2, 10 docs, 0 ms	3,890× fewer
P3	Category + flagged	COLLSCAN, 244,404 docs, 136 ms	Index #4, 826 docs, 1 ms	296× fewer
P4	Prior transactions (pipeline E shape)	`ts` index, 39,135 docs, 62 ms	Index #2, 10 docs, 0 ms	3,914× fewer
P5	Alert feed: High, by time	SORT on COLLSCAN, 1,811 docs	Index #8, 172 docs, no sort	11× fewer
N1	Regex over note text (negative control)	COLLSCAN, 1,811 docs	COLLSCAN, 1,811 docs	None: no index helps
P6	Neo4j: find the seed card	NodeByLabelScan, 8,013 db hits	NodeUniqueIndexSeek, 2 db hits	4,007× fewer

The results support the design. Each rule read completes in constant or logarithmic time, which is why mean latency stays near 17 ms at full rate. Three of the four rules produced no false alerts, as their structural guarantees predict. The index results also show the limit of indexing: the negative control gains nothing, and P1's gain is small because a usable fallback index already existed; the benefit of #10 is that documents examined equal documents returned. Pipeline A's ranking is visibly

led by JEWELRY because the seed data is deliberately skewed so that the chart has a clear leader. Annexure D gives the screenshots and terminal output behind these results.

7.3 Discussion and Limitations

Failed test TC-30. A forced kill lost 33 of 176,157 transactions (0.019%). The stream entries were recovered correctly, but step 10 acknowledges an entry once its document is in the in-memory buffer, up to 500 ms before it reaches MongoDB. Those documents have no second copy to replay from. The system is therefore at-least-once up to the buffer, not up to durable storage. Objective 1's recovery goal is met for entries in the pending list, but "no loss" does not hold for a hard kill.

Other limitations, stated plainly:

- **Redelivery gap.** The idempotency key is set before the work, so a transaction that fails after step 2 is skipped on redelivery rather than retried.
- **Synthetic data.** Thresholds and rates are untested against real card traffic, and rule-based detection is not compared with learned models.
- **Single-node stores.** No clustering or sharding, so results say nothing about horizontal scaling.
- **Injected rings can look larger than six cards.** Normal devices and IPs are legitimately shared by two cards, and the traversal follows them, so a live injection can return more cards than were injected. The isolated seeded ring always shows exactly six.
- **No authentication and no automated unit-test suite.** The analyst identity is a dropdown; verification relies on scripted system tests.
- **Startup latency.** A p99 of 4.4 s across a run that includes startup shows that the system needs a warm-up period.

CHAPTER 8 – CONCLUSION AND FUTURE ENHANCEMENTS

SentinelStream shows that three NoSQL data models can be combined so that each does the job native to it: stream ingestion and sliding windows in Redis, documents, aggregation and triage in MongoDB, and multi-hop ring discovery in Neo4j. The moment that justifies the design is the handoff: Redis notices the third card on one device, and Neo4j returns the whole group in a single traversal, which no individual transaction would reveal.

Table 8.1: Objectives against results

Objective	Result	Status
1. Streaming with at-least-once processing and recovery	Ordered ingestion, idempotent writes, pending entries reclaimed, 0 duplicates; 0.019% of transactions lost on a forced kill	Met, with the stated limitation
2. Explainable real-time scoring on bounded windows	Four rules, mean 17 ms at 150/s, exact evidence on every alert, TTL on every temporary key	Met
3. MongoDB with validation, indexes, CRUD, analytics	Five validated collections, 16 indexes, full CRUD, 5 pipelines, up to 3,914× fewer documents examined	Met
4. Neo4j ring resolution	One bounded traversal, 3–12 ms; 6 of 6 injected cards found	Met
5. Live dashboard and measurement	Five views over WebSocket; throughput, memory and index effectiveness measured	Met

Future enhancements. The most valuable next steps follow directly from the limitations: acknowledge a transaction only after its bulk write succeeds, closing the loss in TC-30; clear the idempotency key on failure, closing the redelivery gap; run several consumers in the consumer group and evaluate sharding on a hashed `cardId`; make the amount rule compare within a merchant category; test against a public card-fraud dataset and compare with a learned model; and add automated tests and real authentication.

REFERENCES

[1] A. Dal Pozzolo, O. Caelen, Y.-A. Le Borgne, S. Waterschoot, and G. Bontempi, "Learned lessons in credit card fraud detection from a practitioner perspective," *Expert Systems with Applications*, vol. 41, no. 10, pp. 4915–4928, 2014, doi: 10.1016/j.eswa.2014.02.026.

[2] A. Dal Pozzolo, G. Boracchi, O. Caelen, C. Alippi, and G. Bontempi, "Credit card fraud detection: A realistic modeling and a novel learning strategy," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 29, no. 8, pp. 3784–3797, 2018, doi: 10.1109/TNNLS.2017.2736643.

[3] F. Carcillo, A. Dal Pozzolo, Y.-A. Le Borgne, O. Caelen, Y. Mazzer, and G. Bontempi, "SCARFF: A scalable framework for streaming credit card fraud detection with Spark," *Information Fusion*, vol. 41, pp. 182–194, 2018, doi: 10.1016/j.inffus.2017.09.005.

[4] V. Van Vlasselaer, C. Bravo, O. Caelen, T. Eliassi-Rad, L. Akoglu, M. Snoeck, and B. Baesens, "APATE: A novel approach for automated credit card transaction fraud detection using network-based extensions," *Decision Support Systems*, vol. 75, pp. 38–48, 2015, doi: 10.1016/j.dss.2015.04.013.

[5] S. Bhattacharyya, S. Jha, K. Tharakunnel, and J. C. Westland, "Data mining for credit card fraud: A comparative study," *Decision Support Systems*, vol. 50, no. 3, pp. 602–613, 2011, doi: 10.1016/j.dss.2010.08.008.

[6] A. Abdallah, M. A. Maarof, and A. Zainal, "Fraud detection system: A survey," *Journal of Network and Computer Applications*, vol. 68, pp. 90–113, 2016, doi: 10.1016/j.jnca.2016.04.007.

[7] L. Akoglu, H. Tong, and D. Koutra, "Graph based anomaly detection and description: A survey," *Data Mining and Knowledge Discovery*, vol. 29, no. 3, pp. 626–688, 2015, doi: 10.1007/s10618-014-0365-y.

[8] R. Cattell, "Scalable SQL and NoSQL data stores," *ACM SIGMOD Record*, vol. 39, no. 4, pp. 12–27, 2011, doi: 10.1145/1978915.1978919.

[9] A. Davoudian, L. Chen, and M. Liu, "A survey on NoSQL stores," *ACM Computing Surveys*, vol. 51, no. 2, pp. 1–43, 2018, doi: 10.1145/3158661.

[10] F. Gessert, W. Wingerath, S. Friedrich, and N. Ritter, "NoSQL database systems: A survey and decision guidance," *Computer Science – Research and Development*, vol. 32, no. 3–4, pp. 353–365, 2017, doi: 10.1007/s00450-016-0334-3.

[11] R. Angles and C. Gutierrez, "Survey of graph database models," *ACM Computing Surveys*, vol. 40, no. 1, pp. 1–39, 2008, doi: 10.1145/1322432.1322433.

[12] K. Kaur and R. Rani, "Managing data in healthcare information systems: Many models, one solution," *Computer*, vol. 48, no. 3, pp. 52–59, 2015, doi: 10.1109/MC.2015.77.

[13] M. Stonebraker, U. Çetintemel, and S. Zdonik, "The 8 requirements of real-time stream processing," *ACM SIGMOD Record*, vol. 34, no. 4, pp. 42–47, 2005, doi: 10.1145/1107499.1107504.

[14] T. Pourhabibi, K.-L. Ong, B. H. Kam, and Y. L. Boo, "Fraud detection: A systematic literature review of graph-based anomaly detection approaches," *Decision Support Systems*, vol. 133, art. no. 113303, 2020, doi: 10.1016/j.dss.2020.113303.

[15] W. Hilal, S. A. Gadsden, and J. Yawney, "Financial fraud: A review of anomaly detection techniques and recent advances," *Expert Systems with Applications*, vol. 193, art. no. 116429, 2022, doi: 10.1016/j.eswa.2021.116429.

[16] A. Cherif, A. Badhib, H. Ammar, S. Alshehri, M. Kalkatawi, and A. Imine, "Credit card fraud detection in the era of disruptive technologies: A systematic review," *Journal of King Saud University – Computer and Information Sciences*, vol. 35, no. 1, pp. 145–174, 2023, doi: 10.1016/j.jksuci.2022.11.008.

[17] F. K. Alarfaj and S. Shahzadi, "Enhancing fraud detection in banking with deep learning: Graph neural networks and autoencoders for real-time credit card fraud prevention," *IEEE Access*, vol. 13, pp. 20633–20646, 2025, doi: 10.1109/ACCESS.2024.3466288.

[18] P. P. Khine and Z. Wang, "A review of polyglot persistence in the big data world," *Information*, vol. 10, no. 4, art. no. 141, 2019, doi: 10.3390/info10040141.

[19] N. Roy-Hubara, P. Shoval, and A. Sturm, "Selecting databases for polyglot persistence applications," *Data & Knowledge Engineering*, vol. 137, art. no. 101950, 2022, doi: 10.1016/j.datak.2021.101950.

[20] J. Monteiro, F. Sá, and J. Bernardino, "Experimental evaluation of graph databases: JanusGraph, Nebula Graph, Neo4j, and TigerGraph," *Applied Sciences*, vol. 13, no. 9, art. no. 5770, 2023, doi: 10.3390/app13095770.

[21] I. Carvalho, F. Sá, and J. Bernardino, "Performance evaluation of NoSQL document databases: Couchbase, CouchDB, and MongoDB," *Algorithms*, vol. 16, no. 2, art. no. 78, 2023, doi: 10.3390/a16020078.

[22] Redis Ltd., "Redis Streams," Redis documentation. [Online]. Available: https://redis.io/docs/latest/develop/data-types/streams/ (accessed Nov. 3, 2026).

[23] Redis Ltd., "Redis sorted sets," Redis documentation. [Online]. Available: https://redis.io/docs/latest/develop/data-types/sorted-sets/ (accessed Nov. 3, 2026).

[24] MongoDB, Inc., "The ESR (Equality, Sort, Range) guideline," MongoDB Manual. [Online]. Available: https://www.mongodb.com/docs/manual/tutorial/equality-sort-range-guideline/ (accessed Nov. 3, 2026).

[25] MongoDB, Inc., "Aggregation operations," MongoDB Manual. [Online]. Available: https://www.mongodb.com/docs/manual/aggregation/ (accessed Nov. 3, 2026).

[26] MongoDB, Inc., "Transactions," MongoDB Manual. [Online]. Available: https://www.mongodb.com/docs/manual/core/transactions/ (accessed Nov. 3, 2026).

[27] Neo4j, Inc., "The Neo4j Cypher Manual." [Online]. Available: https://neo4j.com/docs/cypher-manual/current/ (accessed Nov. 3, 2026).

[28] VMware Tanzu, "Spring Boot reference documentation, version 3.3." [Online]. Available: https://docs.spring.io/spring-boot/3.3/reference/ (accessed Nov. 3, 2026).

ANNEXURE A – REVIEW 1 – PROPOSAL AND PLANNING

Project Title: SentinelStream: Real-Time Card Fraud Detection and Fraud-Ring Discovery Using Polyglot NoSQL Persistence

Batch ID: NS14

Team Name: SentinelStream

Review Date: 17–21 August 2026 (Review–I week in the course schedule)

NoSQL Database: Redis, MongoDB and Neo4j (polyglot persistence)

Review Presentation / Discussion Checklist

☑ Project overview and motivation
☑ Problem statement and significance
☑ Objectives and scope
☑ Literature survey progress and research gap
☑ Proposed solution and expected outcomes
☑ NoSQL database choice and justification
☑ Task allocation and project timeline

Progress Summary

Work Package / Objective	Planned Work	Completed Work	Status
Problem and topic	Topic selection, team formation, problem identification (20–24 July)	Real-time card fraud and fraud-ring discovery chosen; problem statement written; one-member team	Completed
Literature survey	Survey and research gap (27–31 July)	Journal papers on fraud detection, graph methods and NoSQL reviewed; gap identified (Chapter 2)	Completed
Requirements and design	Requirement analysis, proposal, architecture, database selection (3–7 August)	Design specification; polyglot choice of Redis, MongoDB and Neo4j justified; architecture drawn	Completed
Implementation plan	Task allocation and timeline	Vertical-slice build order with the graded core first; all tasks allocated to the sole member	Completed

Key feedback / observations from the review

The points below are the student's own observations at this stage, taken from the project's design notes and decision log.

1. The specification's figures needed reconciling: 150 transactions per second against a placeholder of 1,284 in the mock-up, and a distance of 1,760 km quoted as a speed.
2. At the first-draft population (500 cards, 200 devices), normal traffic would itself trip the fan-out and velocity rules, so the population and pacing were corrected before coding.
3. MongoDB persistence was scheduled too late in the first plan and was moved forward, so that CRUD, indexing and aggregation would be complete first.

Action items before the next review (include responsible member and target date)

1. Set up the environment and initial database implementation. Yash Sehrawat, 28-08-2026.
2. Implement CRUD, validation and core modules. Yash Sehrawat, 04-09-2026.
3. Implement aggregation, indexing and optimisation. Yash Sehrawat, 11-09-2026.

ANNEXURE B – REVIEW 2 – DESIGN AND PARTIAL IMPLEMENTATION

Project Title: SentinelStream: Real-Time Card Fraud Detection and Fraud-Ring Discovery Using Polyglot NoSQL Persistence

Batch ID: NS14

Team Name: SentinelStream

Review Date: 21–25 September 2026 (Review–II week in the course schedule)

NoSQL Database: Redis, MongoDB and Neo4j (polyglot persistence)

Review Presentation / Discussion Checklist

☑ Progress against Review 1 feedback
☑ Refined requirements and system design
☑ Database model / schema and sample data
☑ Implemented modules and features
☑ Screenshots or live demonstration evidence
☑ Queries / CRUD operations / test results, as relevant
☑ Plan to complete implementation and testing

Progress Summary

Work Package / Objective	Planned Work	Completed Work	Status
Environment and ingestion	Environment setup and initial database implementation	Three stores and health check; generator at about 150/s; consumer group with idempotency and acknowledge-last	Completed
CRUD and detection	CRUD, validation and module development	Validators and index registry; four detectors; full triage API; cross-store block-card transaction	Completed
Advanced operations	Aggregation, indexing, optimisation	Pipelines A to E; `explain()` evidence; bulk writer; seed script	Completed
Graph and interface	Partial implementation for demonstration	Neo4j projector and ring queries; ring graph; three live channels; first dashboard	Completed

Key feedback / observations from the review

The points below are the student's own observations at this stage, taken from the project's design notes and decision log.

1. Building pipeline A exposed that `flagged` was always false, because the mapper never received the score; this was fixed where the document is built.
2. Repeated ring injections on one fixed device corrupted the seeded ring (102 cards instead of 6), so each injection now uses a fresh device and IPs.
3. A data wipe while the API was running silently stopped the consumer; it now recovers on its own.

Action items before the next review (include responsible member and target date)

1. Testing, debugging and performance evaluation. Yash Sehrawat, 09-10-2026.
2. Research paper or patent disclosure preparation. Yash Sehrawat, 16-10-2026.
3. Final documentation, GitHub repository, presentation and rehearsal. Yash Sehrawat, 24-10-2026.

ANNEXURE C – REVIEW 3 – FINAL IMPLEMENTATION AND RESULTS

Project Title: SentinelStream: Real-Time Card Fraud Detection and Fraud-Ring Discovery Using Polyglot NoSQL Persistence

Batch ID: NS14

Team Name: SentinelStream

Review Date: 29 October 2026

NoSQL Database: Redis, MongoDB and Neo4j (polyglot persistence)

Review Presentation / Discussion Checklist

☑ Final system overview and completed objectives
☑ Working application demonstration
☑ NoSQL database implementation evidence
☑ Testing methodology, test cases, and results
☑ Performance / result analysis, where relevant
☑ Comparison with objectives and identified limitations
☑ Future enhancements and lessons learned

Progress Summary

Work Package / Objective	Planned Work	Completed Work	Status / Evidence
Final implementation	Complete all modules; working application	Native stack without Docker; light five-view dashboard; four injectors; one-command start and stop	Completed. Figures D.1–D.3
Testing and evaluation	Testing, debugging, performance evaluation	32 test cases, 31 passed; throughput, latency, memory and index effect measured; defects fixed	Completed. Chapter 7
Documentation and demonstration	Report, repository, presentation, demonstration	Final demonstration given on 29 October 2026; repository published; report completed	Completed. Link in Annexure E

Key feedback / observations from the review

The points below are the student's own observations at this stage, taken from the project's design notes and decision log.

1. Running natively exposed a connection defect hidden until then: one Redis connection per transaction exhausted the operating system's ports, failing about 15% of transactions. A connection pool fixed it.
2. Normal traffic raised about 7 false alerts a second because amounts came from the merchant's category; a per-card spending profile reduced this to 0.03%.
3. Capturing evidence showed that declined rows rarely reached the tape and that ring snapshots missed the injected device; both were fixed.

Action items before the document Submission (include responsible member and target date)

1. Re-run the full test suite and record dated evidence. Yash Sehrawat, 03-11-2026. Done.
2. Push the final source code to the GitHub repository. Yash Sehrawat, 03-11-2026. Done.
3. Complete and submit the final report. Yash Sehrawat, 03-11-2026.

ANNEXURE D – IMPLEMENTATION PROOF

Project Title: SentinelStream: Real-Time Card Fraud Detection and Fraud-Ring Discovery Using Polyglot NoSQL Persistence

Batch ID / Team Name: NS14 / SentinelStream

NoSQL Database Used: Redis, MongoDB and Neo4j (polyglot persistence)

Application / Demo URL (if any): http://localhost:3000 (local deployment started with .\run; no public URL)

Attach clear, genuine screenshots or other verifiable evidence. Each item should include a figure number, caption, date, and a brief explanation of what it proves.

Evidence Checklist

☑ Application running / main interface
☑ Core feature(s) demonstrated with sample input and output
☑ NoSQL database connection or configuration (hide passwords, tokens, and other secrets)
☑ Database structure and sample records / documents / keys / graph data
☑ Create, read, update and delete operations, where relevant
☑ Queries or database operations and their outputs
☑ Validation, error handling, or test results
☑ Console / terminal output showing successful execution, where relevant

Evidence Log

Figure No.	Screenshot / Evidence Description	Date	What This Demonstrates
D.1	Live monitor at rest	03-11-2026	Application running: KPI tiles, tape at 150/s, fraud patterns, alert feed
D.2	All four fraud patterns injected	03-11-2026	Core feature, sample input and output: flagged rows, alerts, ring notice
D.3	Alert drawer with two evidence shapes	03-11-2026	Exact, differently shaped evidence stored per rule; `$lookup` context
D.4	Update: status, note, audit trail	03-11-2026	Update with `$push`; audit trail recorded
D.5	Terminal: connections and configuration	03-11-2026	All stores connected; settings shown; password masked
D.6	Terminal: structure and sample data	03-11-2026	Collections, validators, Redis keys and TTLs, graph counts, constraints
D.7	Data stores view	03-11-2026	Live counts in all three stores; buffers with 0 dropped
D.8	Neo4j Browser: seeded ring query	03-11-2026	The graph exists outside the dashboard: 6 cards, 1 device, 2 IPs
D.9	Create: raise a case	03-11-2026	Create, with sample input
D.10	Delete: dismiss with a reason	03-11-2026	Delete with archive copy
D.11	Read: queue New + High	03-11-2026	Read and filter; query plan IXSCAN, 15 keys for 15 returned
D.12	Terminal: CRUD with counts	03-11-2026	Document counts rise and fall with each operation
D.13	Queue with no filter	03-11-2026	COLLSCAN: the same 15 rows without an index
D.14	Live index registry	03-11-2026	All 16 declared indexes in the database; #10 highlighted
D.15	Analytics view	03-11-2026	Aggregation output, each chart labelled with its stages
D.16	Fraud rings view	03-11-2026	Traversal result as a graph; blocked card in red; ring history
D.17	Tape with declined rows	03-11-2026	Cross-store block: blocked cards' payments are declined
D.18	Terminal: queries and outputs	03-11-2026	Aggregation, explain and Cypher outputs from the command line
D.19	Terminal: validation and smoke test	03-11-2026	HTTP 409, validator error 121, smoke checks passing

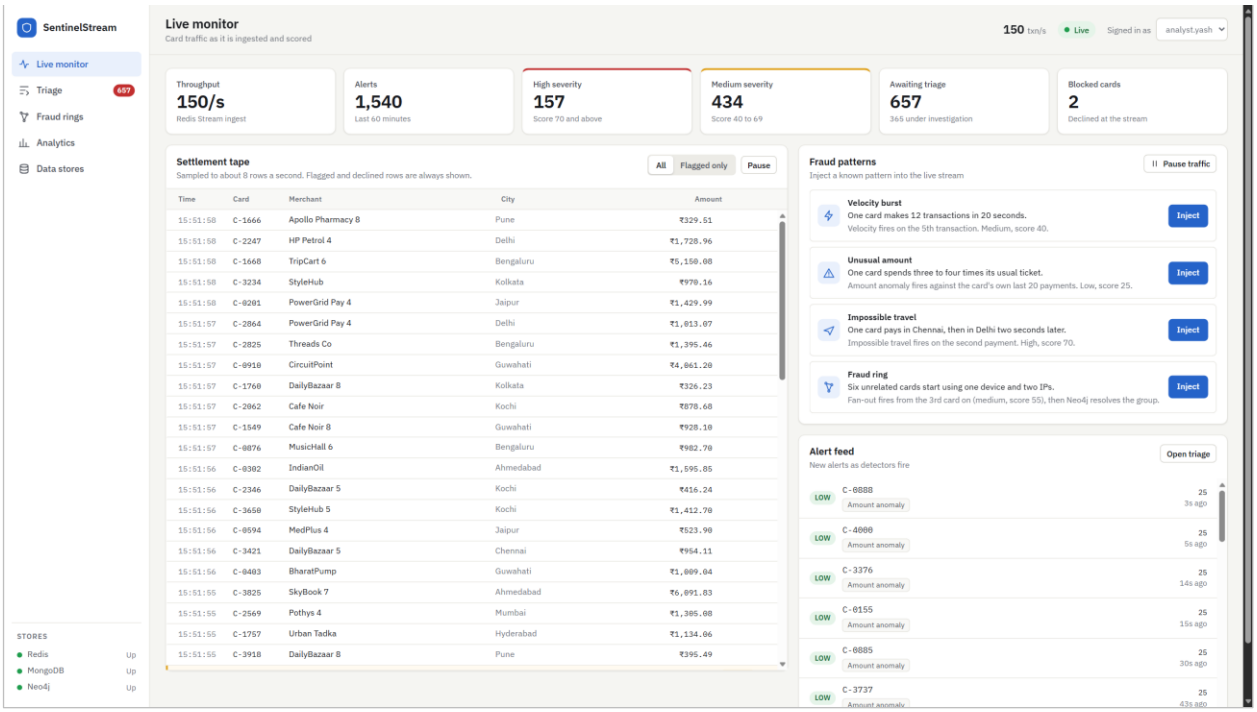


Figure D.1: Live monitor at rest (03-11-2026)

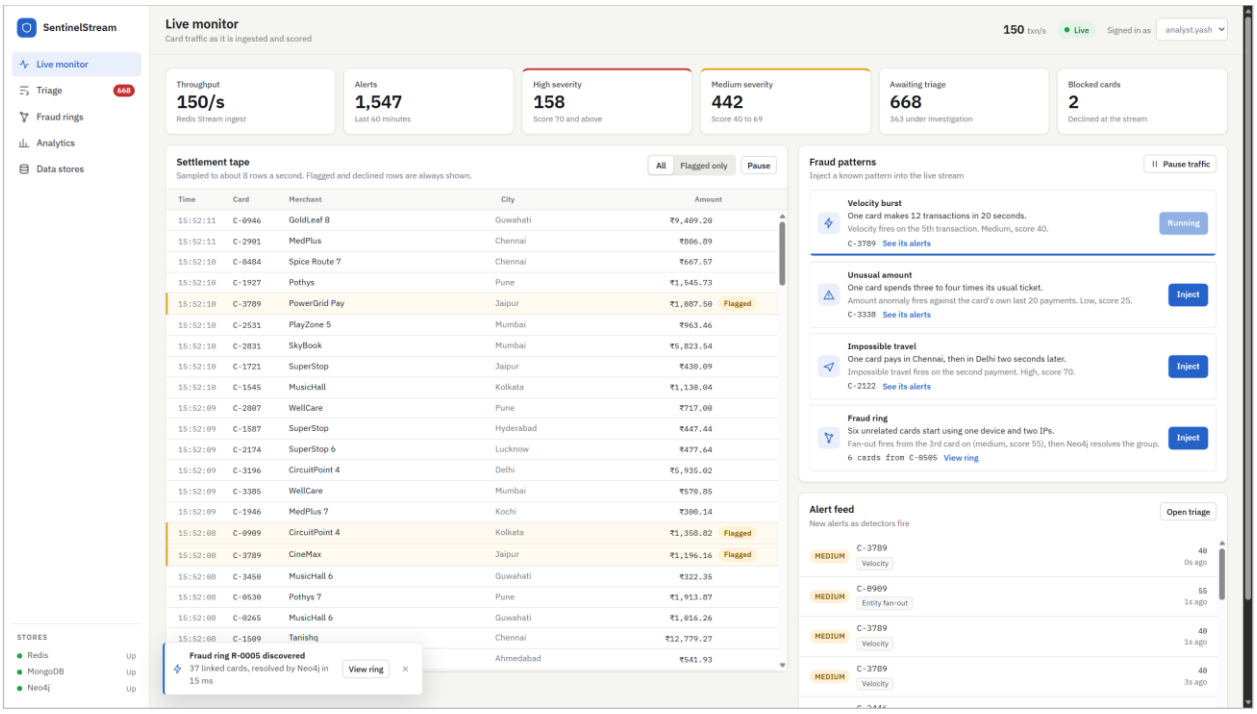


Figure D.2: All four fraud patterns injected (03-11-2026)

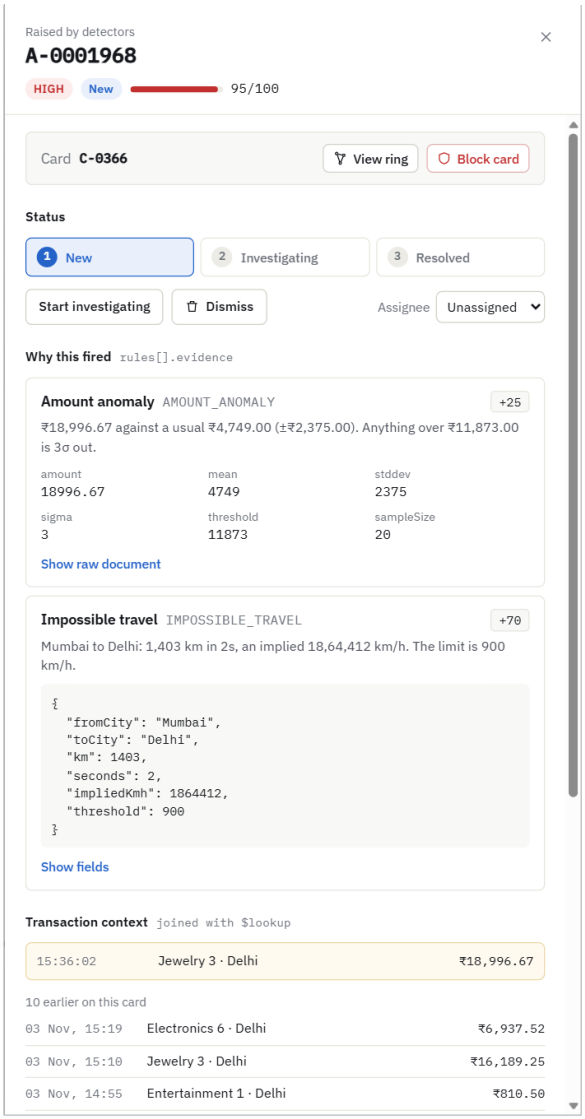


Figure D.3: Alert drawer with two evidence shapes (03-11-2026)

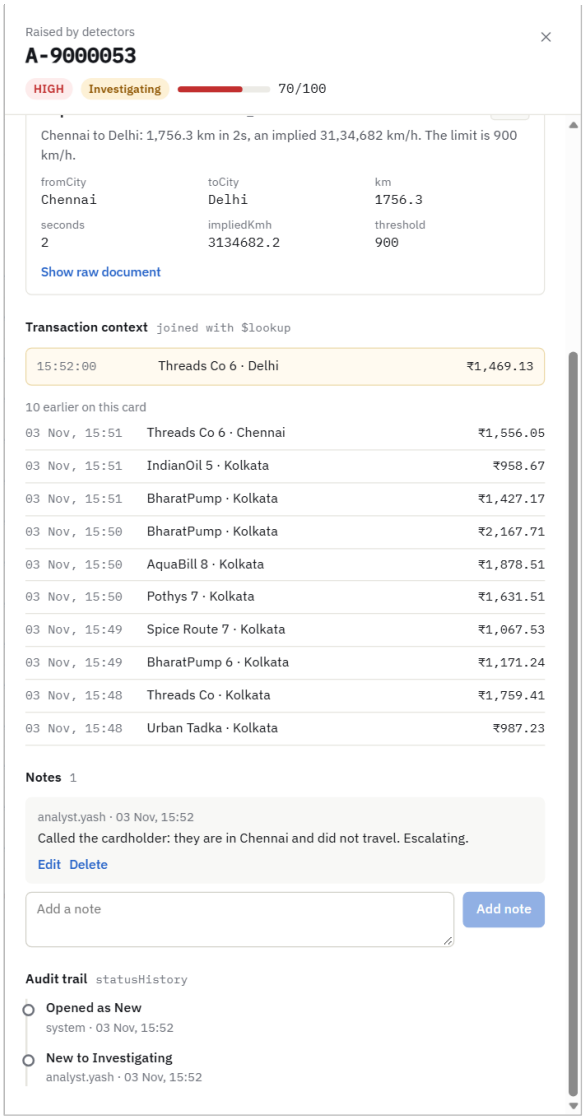


Figure D.4: Update: status, note, audit trail (03-11-2026)

```
> curl -X GET http://localhost:8080/api/health
{"status":"UP","stores":{"redis":"UP","mongo":"UP","neo4j":"UP"}} [HTTP 200]
> redis-cli ping
PONG
> redis-cli config get maxmemory-policy
noeviction
> redis-cli config get appendonly
yes
> mongosh: replica set state
rs0 PRIMARY version 7.0.14
> cypher-shell "CALL dbms.components() YIELD name, versions, edition RETURN name, versions, edition;"
name, versions, edition
"Neo4j Kernel", ["5.20.0"], "community"
> application.yml (connection settings, password masked)
      uri: ${MONGO_URI:mongodb://localhost:27017/sentinel?replicaSet=rs0&directConnection=true&w=majority}
      url: ${REDIS_URL:redis://localhost:6379}
    uri: ${NEO4J_URI:bolt://localhost:7687}
      username: ${NEO4J_USER:neo4j}
      password: ${NEO4J_PASSWORD:********}
```

Figure D.5: Terminal: connections and configuration (03-11-2026)

```
> mongosh: collections, counts, validators
alerts              2001 docs   7 indexes  validator: moderate/error
alerts_archive         3 docs   3 indexes  validator: moderate/error
blocked_cards          2 docs   2 indexes  validator: moderate/error
ring_snapshots         1 docs   2 indexes  validator: moderate/error
transactions       44616 docs   7 indexes  validator: moderate/error
> redis-cli xinfo groups txn:stream
group fraud-detectors  consumers 1  pending 4  entries-read 3821  lag 4
> redis-cli type card:C-0100:txns
zset
> redis-cli ttl card:C-0100:txns
3579
> redis-cli ttl blocklist:cards
-1
> redis-cli smembers blocklist:cards
C-9004, C-0500
> redis-cli dbsize
19150
> cypher-shell "MATCH (n) RETURN head(labels(n)) AS label, count(*) AS nodes ORDER BY label;"
label, nodes
"Card", 3670
"Device", 1990
"Ip", 2669
> cypher-shell "MATCH ()-[r]->() RETURN type(r) AS type, count(*) AS rels ORDER BY type;"
type, rels
"FROM_IP", 3856
"USED_DEVICE", 3757
> cypher-shell "SHOW CONSTRAINTS YIELD name, labelsOrTypes, properties RETURN name, labelsOrTypes, properties;"
name, labelsOrTypes, properties
"card_id", ["Card"], ["cardId"]
"device_id", ["Device"], ["deviceId"]
"ip_addr", ["Ip"], ["ip"]
"merch_id", ["Merchant"], ["merchantId"]
```

Figure D.6: Terminal: structure and sample data (03-11-2026)

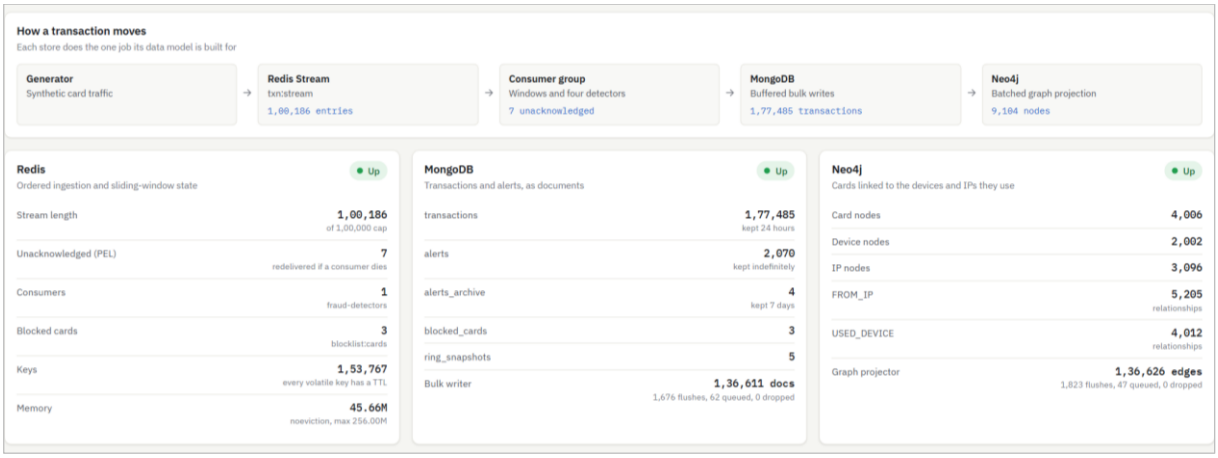


Figure D.7: Data stores view (03-11-2026)

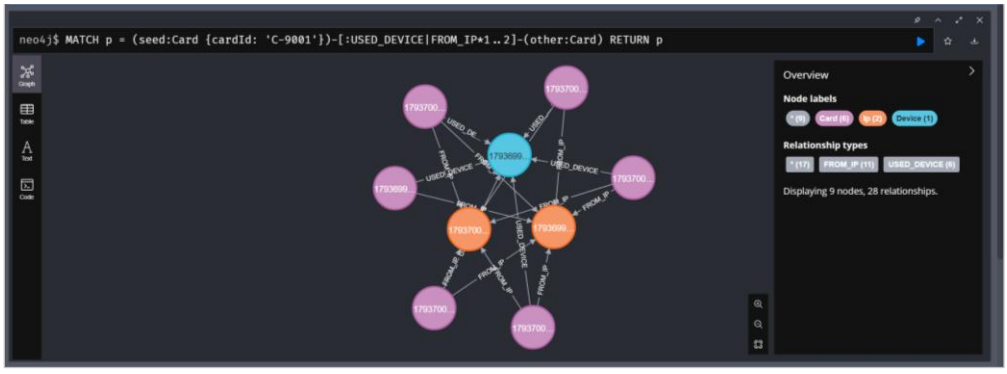


Figure D.8: Neo4j Browser: seeded ring query (03-11-2026)

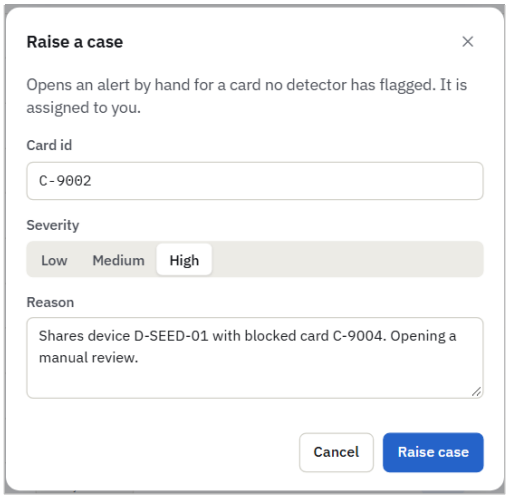


Figure D.9: Create: raise a case (03-11-2026)

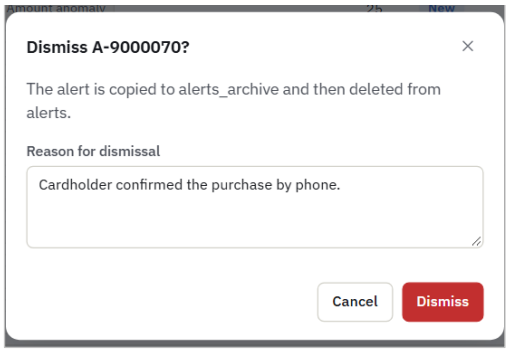


Figure D.10: Delete: dismiss with a reason (03-11-2026)

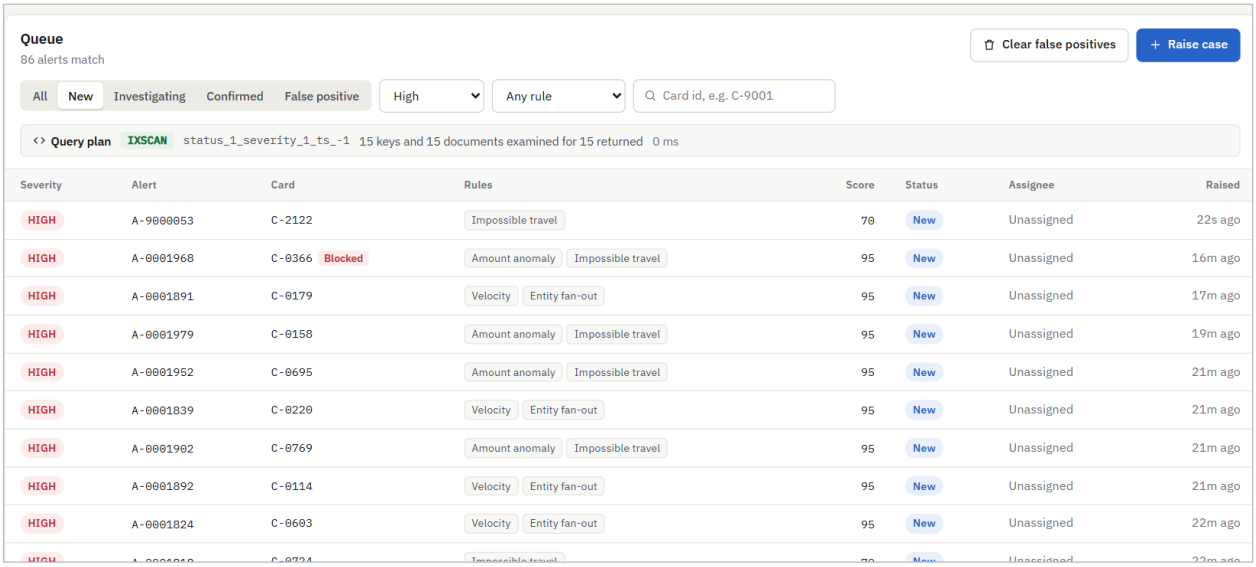


Figure D.11: Read: queue New + High (03-11-2026)

```
> mongosh: counts before
alerts 2002  alerts_archive 3
> curl -X POST http://localhost:8080/api/alerts/manual -d '{"cardId":"C-0042","severity":"MEDIUM","reason":"Report
evidence: manual case","author":"analyst.yash"}'
{"alertId":"A-9000114"} [HTTP 200]
> mongosh: counts after create
alerts 2003  alerts_archive 3
> curl -X PATCH http://localhost:8080/api/alerts/A-9000114/status -d '{"to":"INVESTIGATING","by":"analyst.yash"}'
status updated [HTTP 200]
> curl -X POST http://localhost:8080/api/alerts/A-9000114/notes -d '{"author":"analyst.yash","body":"Checked BIN
range."}'
note added [HTTP 200]
> mongosh: db.alerts.findOne({alertId:'A-9000114'})
status INVESTIGATING  statusHistory 2  notes 1  origin ANALYST
> curl -X PATCH http://localhost:8080/api/alerts/A-9000114/status -d
'{"to":"FALSE_POSITIVE","by":"analyst.yash","resolution":"Purchase confirmed."}'
resolved [HTTP 200]
> curl -X DELETE http://localhost:8080/api/alerts/A-9000114?reason=Report+evidence&by=analyst.yash
{"alertId":"A-9000114","dismissed":true} [HTTP 200]
> mongosh: counts after delete
alerts 2002  alerts_archive 4
```

Figure D.12: Terminal: CRUD with counts (03-11-2026)

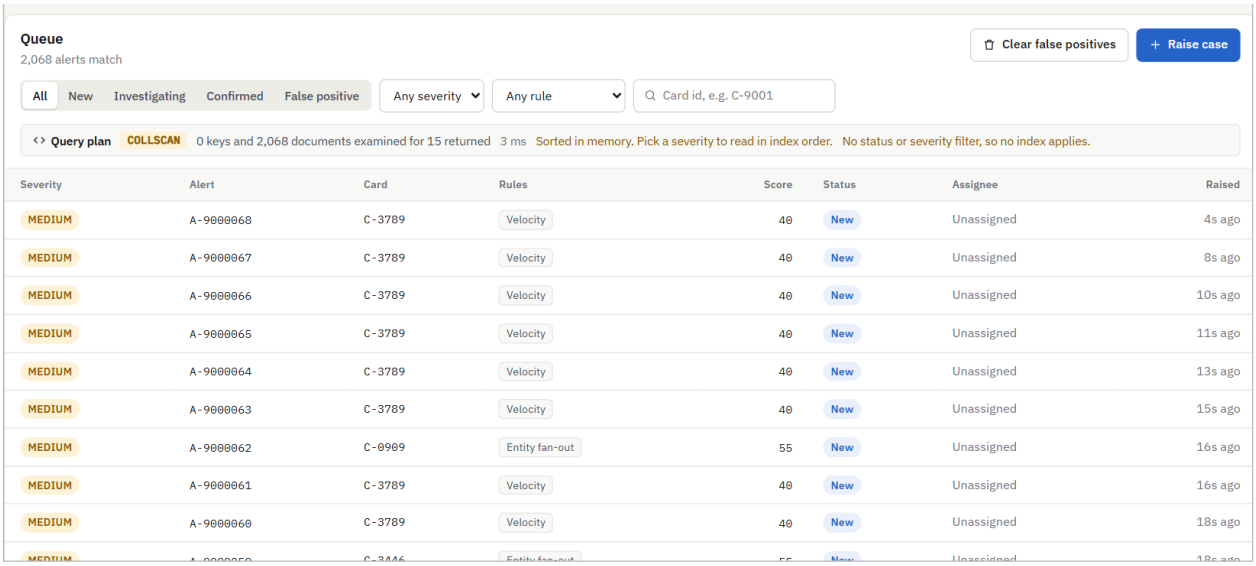


Figure D.13: Queue with no filter (03-11-2026)

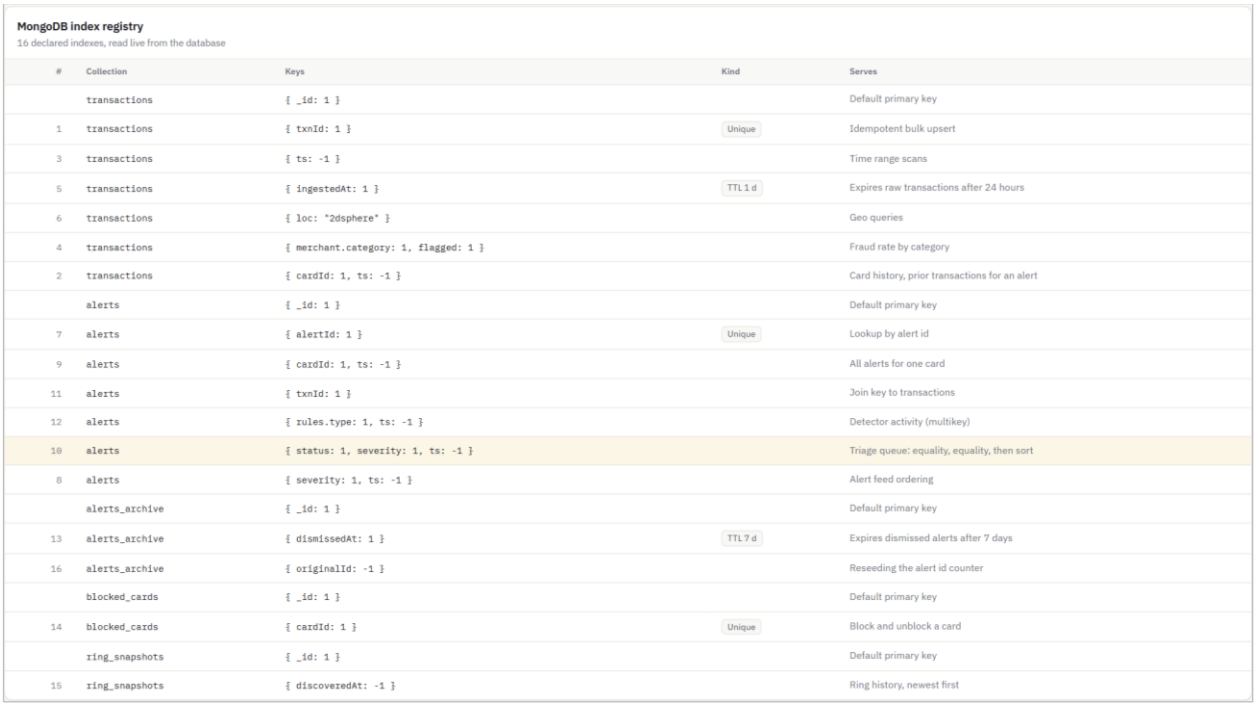


Figure D.14: Live index registry (03-11-2026)

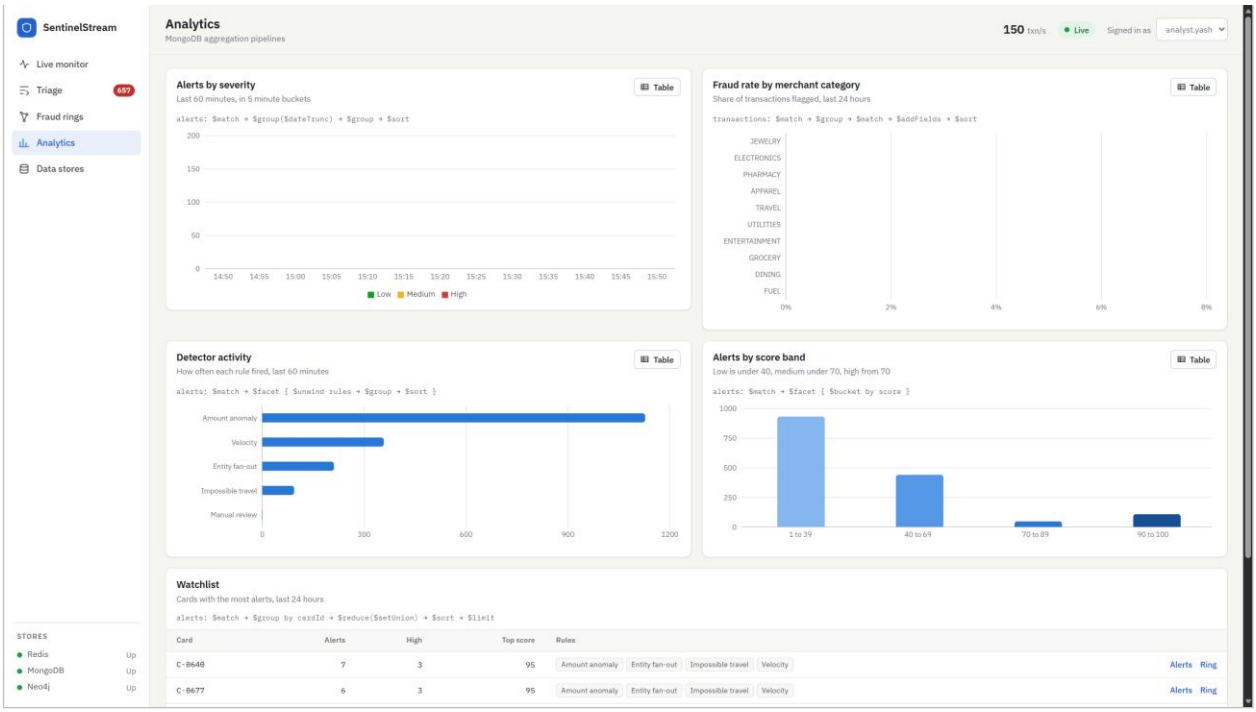


Figure D.15: Analytics view (03-11-2026)

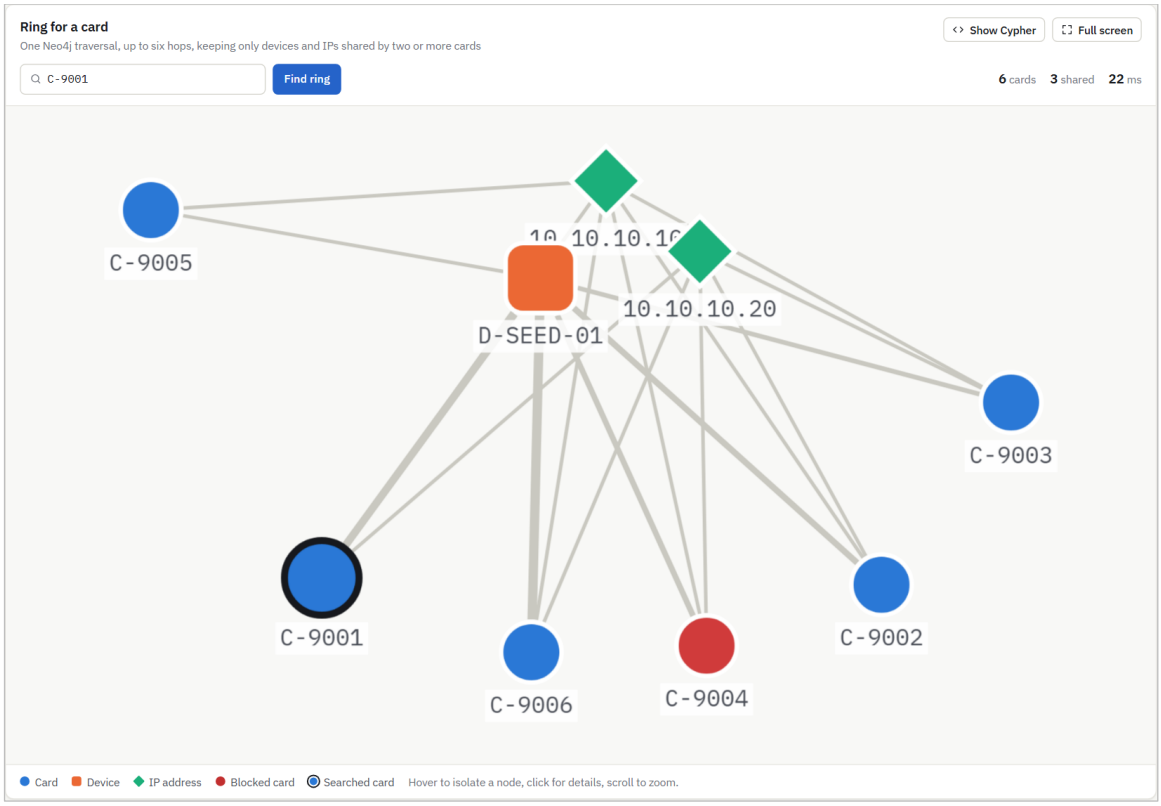


Figure D.16: Fraud rings view (03-11-2026)

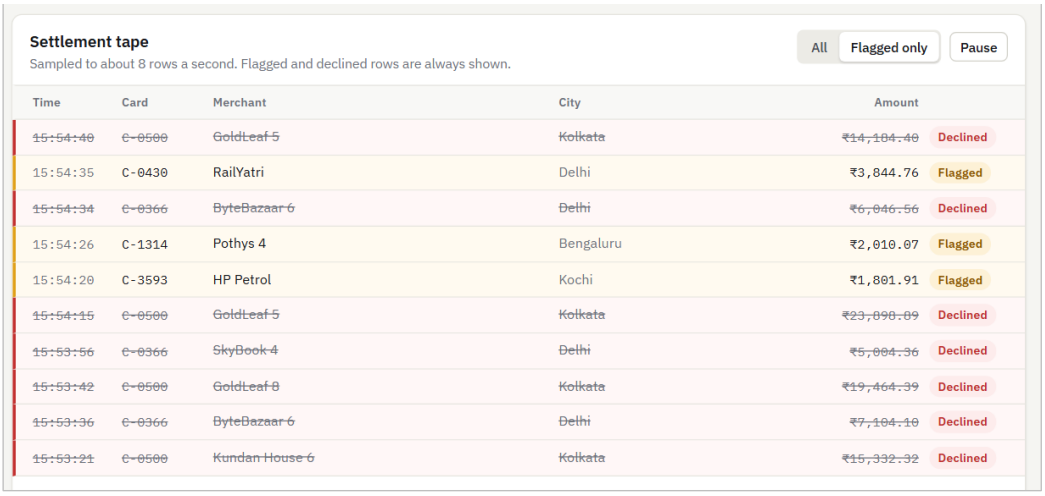


Figure D.17: Tape with declined rows (03-11-2026)

```
> mongosh: pipeline A, fraud rate by category (top 3)
JEWELRY       total 5073 flagged 800 fraudRate 0.1577
PHARMACY      total 4992 flagged 135 fraudRate 0.027
ELECTRONICS   total 9611 flagged 250 fraudRate 0.026
> mongosh: explain of the triage queue (status NEW, severity HIGH, newest first, limit 15)
LIMIT <- FETCH <- IXSCAN(status_1_severity_1_ts_-1)
keysExamined 15 docsExamined 15 nReturned 15
> curl -X GET http://localhost:8080/api/rings/C-9001
found=true queryMs=10 [HTTP 200]
> cypher-shell "MATCH (seed:Card {cardId:'C-9001'})-[:USED_DEVICE|FROM_IP*1..6]-(m:Card) WHERE m <> seed RETURN
collect(DISTINCT m.cardId) AS others;"
others
["C-9004", "C-9005", "C-9002", "C-9006", "C-9003"]
```

Figure D.18: Terminal: queries and outputs (03-11-2026)

```
> curl -X PATCH http://localhost:8080/api/alerts/A-9000001/status -d
'{"to":"FALSE_POSITIVE","by":"analyst.yash","resolution":"x"}'
{"timestamp":"2026-11-03T10:30:40.614+00:00","status":409,"error":"Conflict","message":"Illegal transition NEW ->
FALSE_POSITIVE","path":"/api/alerts/A-9000001/status"} [HTTP 409]
> mongosh: db.alerts.insertOne({alertId:'BAD', severity:'CRITICAL', ...})
MongoServerError: code 121, Document failed validation
> .\scripts\smoke.ps1
health                      OK
create (manual alert)       OK  -> A-9000115
update (status transition)  OK
delete (dismiss)            OK
aggregation (kpi facet)     OK
explain (index #10 live)    OK  status_1_severity_1_ts_-1, 20 keys / 20 docs
ring (seeded C-9001)        OK  6 cards, 3ms

All checks passed.
```

Figure D.19: Terminal: validation and smoke test (03-11-2026)

Implementation Summary (what is completed and what remains)

Completed: the generator and stream consumer; four detectors; all three data stores with validators, 16 indexes and 4 constraints; full CRUD triage; five aggregation pipelines; ring discovery; the five-view dashboard; one-command start and stop; and the test suite. **Remains:** closing the small loss on a forced kill and the redelivery gap (Section 7.3), and the future enhancements in Chapter 8. All figures were taken from the running system on the date shown; no passwords or personal data appear in them.

Important: Do not include passwords, API keys, access tokens, personal data, or confidential information in screenshots or the repository.

ANNEXURE E – GitHub Repository and Code Submission Details

Project Title: SentinelStream: Real-Time Card Fraud Detection and Fraud-Ring Discovery Using Polyglot NoSQL Persistence

Batch ID / Team Name: NS14 / SentinelStream

GitHub Repository URL: https://github.com/sehrawaty/SentinelStream

NoSQL Database: Redis, MongoDB and Neo4j (polyglot persistence)

Application / Demo URL: http://localhost:3000 (local deployment started with .\run; no public URL)

Repository Checklist

☑ Repository link opens and is accessible to the faculty reviewer
☑ Source code is uploaded and organised into meaningful folders
☑ README includes project overview, features, setup instructions, and run instructions
☑ Requirements / dependency list is included
☑ Database setup instructions and sample / seed data are provided where possible
☑ Example configuration file is provided without secrets
☑ Screenshots or demo instructions are included
☑ Team contributions are documented
☑ No passwords, API keys, tokens, private datasets, or confidential information are committed
☑ Repository contains only material the team is authorised to share

The only credential in the repository is a local development default for Neo4j, which binds to 127.0.0.1 only. It is not a production secret. The repository holds no API keys, tokens or personal data, and all data is synthetic.

How to run the project (commands / steps)

```
git clone https://github.com/sehrawaty/SentinelStream.git
cd SentinelStream
powershell -ExecutionPolicy Bypass -File scripts\native\setup-native.ps1   # once
.\run      # stops any earlier run, starts everything on freshly seeded data, opens http://localhost:3000
.\stop     # stops everything
```

Database setup and sample-data instructions

`setup-native.ps1` downloads Redis, MongoDB and Neo4j as portable builds, with no installation or administrator rights. `.\run` initiates the replica set `rs0`, creates the 16 indexes and 4 constraints, and loads the seed data through `scripts/seed.mjs --wipe`: 50,040 transactions, 2,002 alerts and an isolated six-card ring on `C-9001`. `.env.example` lists the optional settings. `scripts/smoke.ps1` then checks the graded path, and `scripts/explain-evidence.mjs` regenerates the index evidence.

Member contributions (name, register number, contribution)

Yash Sehrawat, 23BCE0161: sole team member, responsible for requirements, system design, the data models for all three stores, the API, stream consumer and detectors, the dashboard, testing and documentation.

ANNEXURE F – Research paper draft or patent disclosure draft (Not Mandatory)

Not included. This annexure is optional.